



Computational Physics: Introductory Course

Spring 2026

Project Report:

Atomic Structure

Name: Cecilia Lis Costa

Personal number/National Identification Number: 19880129 4227

Hand-in date: 2026-05-22

Introduction

This project will explore part of the atomic structure, more specifically, the behavior of electrons around the atomic nucleus. This behavior, as well as of any non-relativistic particle, can be described by Schrödinger's equation.

$$\hat{H}\Psi = E\Psi$$

The expression correlates the total energy of a system (potential plus kinetic energy), expressed through the Hamiltonian operator $\hat{H}$, with a constant value E . The wavefunction Ψ translates how a particle, or a system of particles, behaves in terms of time and space. Associating a particle's behavior to a wave function essentially differs how classical systems model problems involving particles. In the Quantum Mechanics paradigm, a particle can no longer be described as a point in the space, it can no longer have a specific position, instead been associated to a probability of being found within a region. The interaction between particles cannot be described as collisions, but in terms of interference of their respective wavefunction.

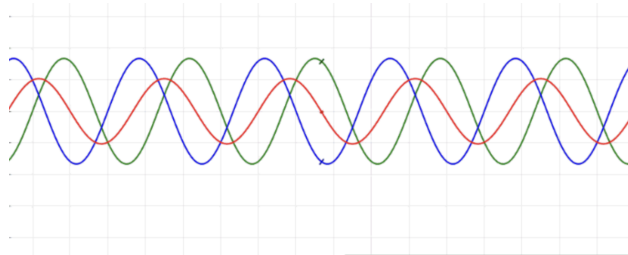


Figure 1: Green and blue lines representing different particle's wave functions and red, the resulting wavefunction from the particle's interference [Wik, Wikipedia - Wave interference]

Quantum mechanics is capable of modelling problems which classical systems cannot. Experiments with particles which contradict classical models, can then be understood when analyzed under Quantum Mechanics lenses. Some such scenarios will be discussed further in the context of the problems proposed in this project. In regard to the solution of those problems, some scenarios posed by the problems, allow developing analytical solutions when applied to Schrödinger's equation. Those are usually specific scenarios under special conditions which greatly simplify the expression obtained. In most real scenarios, the system is too complex, involving many particles and non-uniform potential. For such systems, techniques to obtaining satisfactory solutions were devised and improved along the time. The project description provides the details and implementation of one such technique, Cowell-Numerov method. Which we will be using when applicable to such problems and modify it to better suit specific scenarios.

Theoretical background

The wave function of a particle is the model through which we can, more accurately, describe the properties and behavior of a particle today. Among those, the probability of finding a particle in a certain region, its velocity, energy, or how it interacts with other particles and the region's potential. When solving Schrödinger's equation, we are looking for solution pairs (E, Ψ) which satisfy the equation or system of equations obtained from the expression:

$$\hat{H}\Psi = E\Psi \rightarrow \\ -\frac{\hbar^2}{2m}\nabla^2\Psi + V\Psi = E\Psi$$

Where E will be an eigenvalue solution for the systems obtained from applying the Hamiltonian operator $\hat{H}$, and Ψ , the eigenstate solution. The Hamiltonian operator provides an expression for the system's total energy, where the factor $-\frac{\hbar^2}{2m}\nabla^2$ represents the associated kinetic energy and V , the expression for the potential in the region being analyzed.

Mathematically speaking, E and Ψ , as mentioned, are the eigenvalue and the corresponding eigenstate for the system's solution. The physical meaning of E and Ψ however, are the total energy of the particle, or system of particles being represented, and the wave-function associated, in a given state.

The different pairs of solutions obtained for this equation expresses the different energy levels a particle can assume in an atomic system. For an electron in an atom of Hydrogen for example, it is translated into the different states expressed by the combination of the atomic numbers n, l, m_l, m_s .

Beyond very simple systems such as for the Hydrogen, the expression expanded from Schrödinger's equation became extremely complex. As co-interactions of many particles' among each other must be taken into account. Solving such problems analytically became unfeasible, and as mentioned before, convenient computational methods can be applied, providing satisfactory approximations.

On the following sessions, we will be solving the proposed problems, and according to each, providing, when applicable, additional context use to understand and find the respective solution. When relevant, we will also explore further and highlight any particularly interesting observation which can be made based on the exercise scenarios.

Results and Discussions

On this exercise, we are required to plot the analytical solutions for Schrödinger Radial equation for a few given states on an electron in an atom of Hydrogen:

The Hydrogen atom embodies specific case with characteristics which enables solving Schrödinger's radial equation analytically:

- Central potential: Allows separation of radial and angular components of the equation due to spherical symmetry.

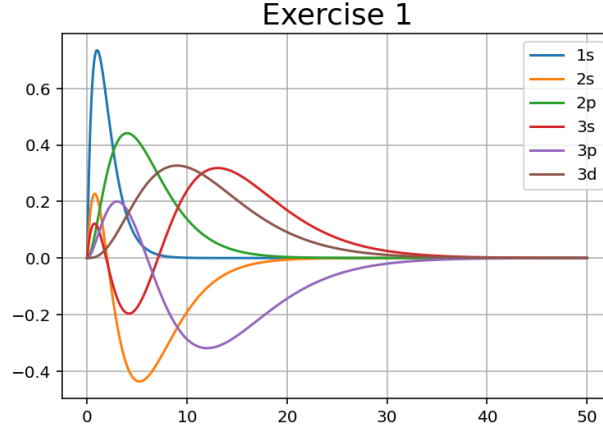


Figure 2

- Single electron: Systems with multiple electrons demands accounting for the influence exert on each other in terms of potential.
- Coulomb's field: Arises from the nuclei, and the potential expression can be written as $V(r) = -\frac{Z}{r}$, Z being the number of protons on the nuclei, with $Z = 1$, for Hydrogen.

The radial expressions $P(r)_{nl}$, plot on Figure 1, include the states: 1s, 2s, 2p, 3s, 3p, 3d.

In a non-excited state, the hydrogen single electron will occupy the lowest energy level, 1s. On this state, the radial function would, conceptually, shape a sphere. The "conceptually" qualifier here is due to the fact the wave function Ψ , does not define a physical property such as size or location. $|\Psi|^2$, however, defines the probability of finding the electron on a relative location. And so, the conceptual mathematical wave-function shape Ψ , will have a correlation with the probability density $|\Psi|^2$.

From the plot on Figure 1, we can observe the radial wave-function $P(r)_{1s} > 0$, has no nodes (does not cross from positive to negative values or vice-versa). On $P(r)_{2s}$ we can observe one node. The function $P(r)_{2s} > 0$ for r parting from the nuclei, crossing the 0 value and assuming negative values from there to $r \rightarrow \infty$, where $P(r)_{2s} \rightarrow 0$. These observations are consistent with figure 2, which shows the probability density around the nuclei, with the phase shift, represented with different colors. Here the phase shift is what we observe when the wave function crosses to negative values. This has no effect on the actual value of the probability density given by $|\Psi|^2 = |P(r)|^2$, as $|P|^2 > 0$, for any given r .

Exercise 2

The exercise requires plotting the radial function for the state 1s, given by:

$$P_{1s}(r) = 2Z^{2/3}re^{-\frac{Z}{r}}$$

For different values of $Z = 1, 2, 3, 4$.

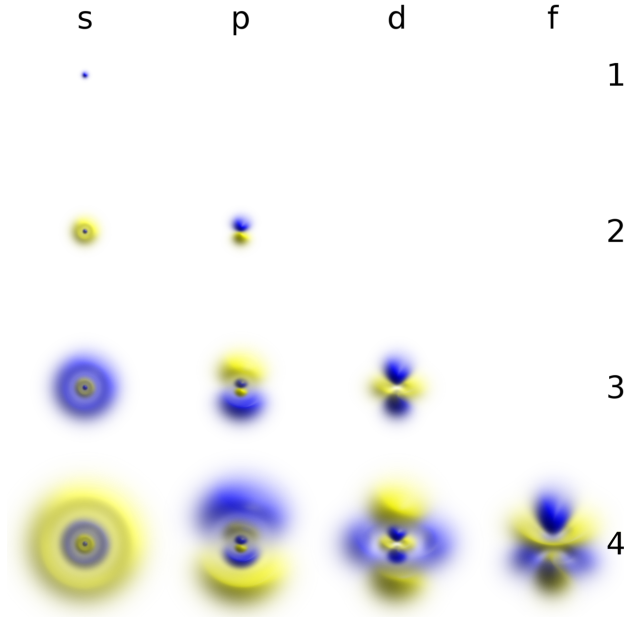


Figure 3

Previously, we discussed the electric field on the Hydrogen follows the Coulomb's field and the potential can be expressed as $V(r) = -\frac{Z}{r}$. Increasing the value of Z , increases the potential well depth in which the electron is placed. Schrödinger's equation, relates the potential and kinetic energy of the electron to the total energy of the system, given by the eigenvalue E , which is constant. It means, in regions in which the potential magnitude is greater, the kinetic must be smaller, so the total energy remains constant. Consequently, the wave-function "presence" must fade quicker in regions further for the nucleus, with a faster conversion rate of kinetic into potential energy.

That is exactly what the Figure 3 plot for the radial function $P_{ls}(r)$, with different values of Z , shows. For greater Z , the wave function converges to zero much sooner. The wave function intensity near the nucleus have also a higher peak. It means, for the probability density $|P_{ls}(r)|^2$, the probability to find the electron on those regions increases.

Exercise 3

As a first question, the exercises pose:

What is meant by the radial density function $D(r)$ and how do you interpret it?

According to the project description, the radial density function is the probability of finding the electron at a given distance r from the nucleus. This probability is given by the probability density function integration over the angular part of the expression:

$$D(r) = \int_0^{2\pi} \int_0^\pi |\Psi(r, \theta, \phi)|^2 \sin \theta d\theta d\phi$$

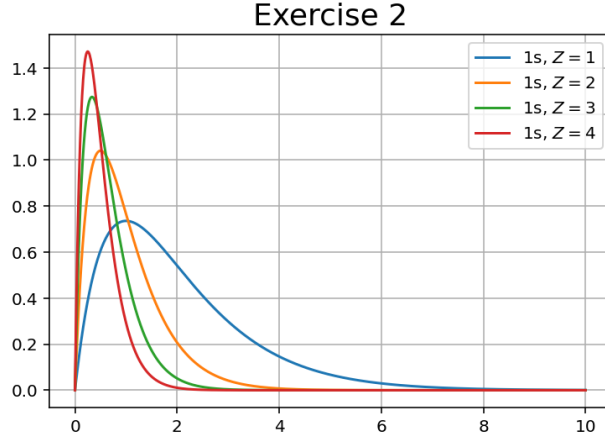


Figure 4

And assuming a central potential, $Y(\theta, \phi)$ is symmetrical around z axis, therefore, the angular and radial probability densities are independent of each other.

$$\int_0^{2\pi} \int_0^\pi |P(r)Y(\theta, \phi)|^2 \sin \theta d\theta d\phi = |P(r)|^2 \int_0^{2\pi} \int_0^\pi |Y(\theta, \phi)|^2 \sin \theta d\theta d\phi$$

And finally, integrating the angular probability density over the entire region will result in 1.

$$|P(r)|^2 \int_0^{2\pi} \int_0^\pi |Y(\theta, \phi)|^2 \sin \theta d\theta d\phi = |P(r)|^2$$

One way to visualize the meaning of this definition is to interpret as the probability of finding the electron on a region defined by a shell at a distance r around the nucleus.

The exercise follow up this question by asking to plot $D(r)$ for $3s$, $3p$, $3d$ and $Z = 1$.

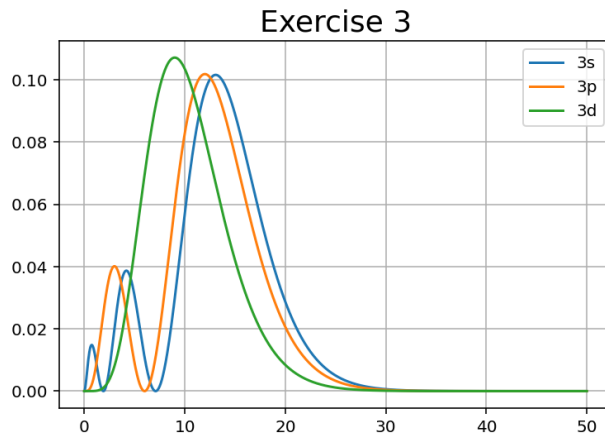


Figure 5

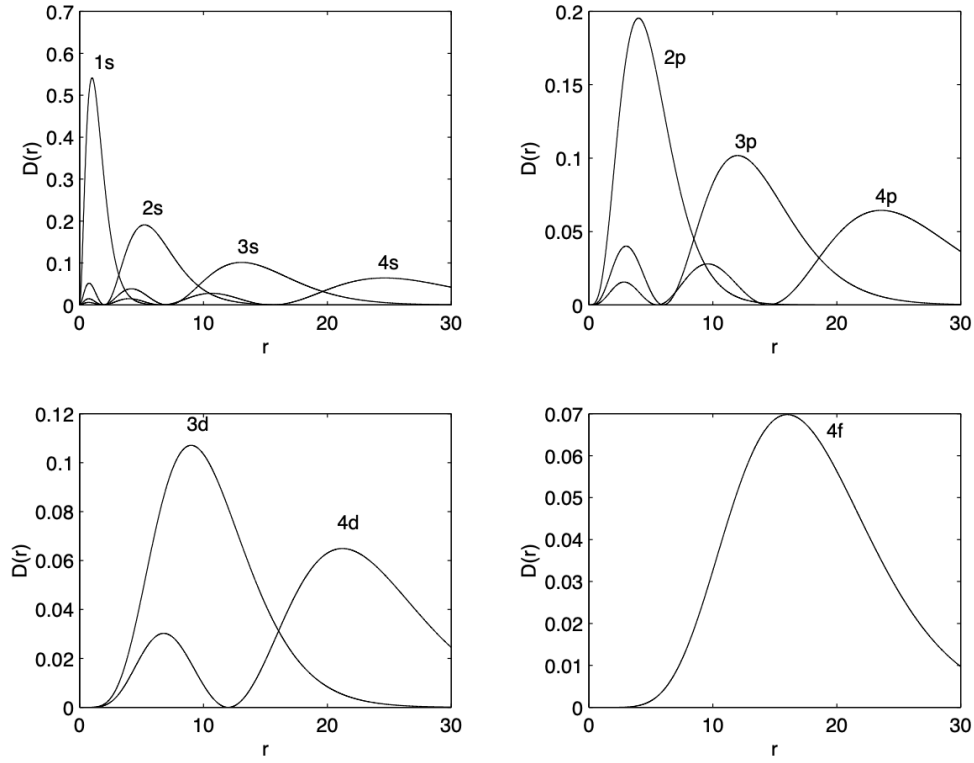


Figure 6: Radial charge densities for hydrogen. Note that only s electrons have finite charge densities at the origin and that other orbitals are kept away from the nucleus by the $\frac{l(l+1)}{2r^2}$ term in the radial part of the SE, compare also exercise 9.

Comparing the two figures, it is possible to verify the plot from Figure 5 is consistent with the expected results. All 3 states peak around 0.1, converge to zero around the same interval and have the same number of nodes.

Exercise 5

This problem requires verifying the given sphere harmonics follow the orthogonality relation programmatically, using the library *sympy*.

To verify this condition, we must proof:

$$\langle Y_{ml}, Y_{m'l'} \rangle = \delta_{ll'} \delta_{mm'}$$

What means to say:

$$\begin{cases} l \neq l', \delta_{ll'} = 0 \\ l = l', \delta_{ll'} = 1 \end{cases} \quad (1)$$

and,

$$\begin{cases} m \neq m', \delta_{mm'} = 0 \\ m = m', \delta_{mm'} = 1 \end{cases} \quad (2)$$

And so,

$$\begin{cases} m = m' \text{ and } l = l', \langle Y_{ml}, Y_{m'l'} \rangle = 1 \\ \text{else, } \langle Y_{ml}, Y_{m'l'} \rangle = 0 \end{cases} \quad (3)$$

The expression $\langle Y_{ml}, Y_{m'l'} \rangle$ is the inner product between two functions, and analogous to the inner product (or dot product) for vectors, when the two functions are orthogonal to each other, the projection of the function space over the other is null.

$Y_{00}(\theta, \phi)$ Orthogonality check against:

$[Y_{00}(\theta, \phi), Y_{1+1}(\theta, \phi), Y_{1-1}(\theta, \phi), Y_{10}(\theta, \phi), Y_{2+2}(\theta, \phi), Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$

$$\begin{aligned} \langle Y_{00}(\theta, \phi), Y_{00}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \frac{\sin(\theta)}{4\pi} d\theta d\phi = 1 \\ \langle Y_{00}(\theta, \phi), Y_{1+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{-\sqrt{6} e^{i\phi} \sin^2(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{1-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{\sqrt{6} e^{-i\phi} \sin^2(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{10}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{\sqrt{3} e^{2i\phi} \sin(\theta) \cos(\theta)}{4\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2+2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{\sqrt{30} e^{2i\phi} \sin^3(\theta)}{16\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2-2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{\sqrt{30} e^{-2i\phi} \sin^3(\theta)}{16\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{-\sqrt{30} e^{i\phi} \sin^2(\theta) \cos(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{\sqrt{30} e^{-i\phi} \sin^2(\theta) \cos(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^{\pi} \left(\frac{\sqrt{5} (-\sin^2(\theta) + 2\cos^2(\theta)) \sin(\theta)}{8\pi} \right) d\theta d\phi = 0 \end{aligned}$$

Figure 7: Orthogonality verification for Y_{00} against the whole given set of spherical harmonics. Note the integral evaluates to 1 only for the inner product of Y_{00} against itself.

The remaining verifications will be included on the appendix.

Exercise 6

The exercise requires deriving the expression from which the equality condition for the energy of a particle at the classical turning point, is translated on Cowell-Numerov's method. The condition is applied at a distance of r_c from the nucleus, representing the classical turning point, meaning, where the potential energy $V(r) > E$, with E being the total energy of the particle.

This condition established that the radial function $P(r)$ and its derivative, must be continuous at r_c . This implies, the expression for $P_1(r)$, with $r \in [0, r_c]$, must equal $P_2(r)$, with $r \in [r_c, \infty]$, as well as their respective derivatives.

The condition is expressed as:

$$2(E_1 - E_2)P_1(r)P_2(r) = \frac{d}{dr} \left(\frac{dP_2(r)}{dr}P_1(r) - \frac{dP_1(r)}{dr}P_2(r) \right) \quad [1]$$

$$2(E_1 - E_2) \int_{r_c}^{\infty} P_1(r)P_2(r) = \left[\frac{dP_2(r)}{dr}P_1(r) - \frac{dP_1(r)}{dr}P_2(r) \right]_{r_c}^{\infty} = \left[\frac{dP_2(r)}{dr}P_1(r) - \frac{dP_1(r)}{dr}P_2(r) \right]^+ \quad [2]$$

$$2(E_1 - E_2) \int_0^{r_c} P_1(r)P_2(r) = \left[\frac{dP_2(r)}{dr}P_1(r) - \frac{dP_1(r)}{dr}P_2(r) \right]_0^{r_c} = \left[\frac{dP_2(r)}{dr}P_1(r) - \frac{dP_1(r)}{dr}P_2(r) \right]^- \quad [3]$$

From the radial Schrödinger's equation, we can write the expression for the energies E_1 and E_2 at r_c as:

$$\frac{d^2P_1(r_c)}{dr_c^2} + 2 \left(E_1 - V(r_c) + \frac{l(l+1)}{2r_c^2} \right) P_1(r_c) = 0 \quad [4]$$

And

$$\frac{d^2P_2(r_c)}{dr_c^2} + 2 \left(E_2 - V(r_c) + \frac{l(l+1)}{2r_c^2} \right) P_2(r_c) = 0 \quad [5]$$

Since both expression have the same form, we can perform a few operations on the first and assume we replicate it on the second, just so to write it in a form more convenient for the derivation.

Taking the first expression for E_1 and shifting the second derivative term to the right side, then dividing both sides by $P_1(r_c)$, we obtain:

$$2 \left(E_1 - V(r_c) + \frac{l(l+1)}{2r_c^2} \right) = -\frac{d^2P_1(r_c)}{dr_c^2} \frac{1}{P_1(r_c)} \quad [6]$$

By performing the same transformations on the expression for E_2 , then subtracting both expressions:

$$2 \left(E_1 - V(r_c) + \frac{l(l+1)}{2r_c^2} \right) - 2 \left(E_2 - V(r_c) + \frac{l(l+1)}{2r_c^2} \right) = -\frac{d^2 P_1(r_c)}{dr_c^2} \frac{1}{P_1(r_c)} - \left(-\frac{d^2 P_2(r_c)}{dr_c^2} \frac{1}{P_2(r_c)} \right) \quad [7]$$

For both expressions, the terms for the potential $V(r)$ and angular momentum dependent $\frac{l(l+1)}{2r_c^2}$ are the same, so they cancel each other on the left side. Let's also adjust the sign for the second term on the right side:

$$2(E_1 - E_2) = -\frac{d^2 P_1(r_c)}{dr_c^2} \frac{1}{P_1(r_c)} + \frac{d^2 P_2(r_c)}{dr_c^2} \frac{1}{P_2(r_c)} \quad [8]$$

Multiplying both sides by $P_1(r_c)P_2(r_c)$:

$$2(E_1 - E_2) P_1(r_c)P_2(r_c) = -\frac{d^2 P_1(r_c)}{dr_c^2} P_2(r_c) + \frac{d^2 P_2(r_c)}{dr_c^2} P_1(r_c) \quad [9]$$

The left side already has the same form as on [1]. For the right side we want to obtain the following expression:

$$\frac{d}{dr_c} \left(\frac{dP_2(r_c)}{dr_c} P_1(r_c) - \frac{dP_1(r_c)}{dr_c} P_2(r_c) \right)$$

Applying the product rule for derivatives for each individual term on the form $\frac{d}{dr} \left(\frac{dP_i(r)}{dr} P_j(r) \right)$:

$$\begin{aligned} -\frac{d}{dr_c} \left(\frac{dP_1(r_c)}{dr_c} P_2(r_c) \right) &= -\frac{d^2 P_1(r_c)}{dr_c^2} P_2(r_c) - \frac{dP_2(r_c)}{dr_c} \frac{dP_1(r_c)}{dr_c} \\ \frac{d}{dr_c} \left(\frac{dP_2(r_c)}{dr_c} P_1(r_c) \right) &= \frac{d^2 P_2(r_c)}{dr_c^2} P_1(r_c) + \frac{dP_1(r_c)}{dr_c} \frac{dP_2(r_c)}{dr_c} \end{aligned}$$

Note that the second term on both expression $\frac{dP_2(r_c)}{dr_c} \frac{dP_1(r_c)}{dr_c}$, is the same, with inverted sign.

And by summing up we obtain the exact left side of [9]:

$$-\frac{d^2 P_1(r_c)}{dr_c^2} P_2(r_c) + \frac{d^2 P_2(r_c)}{dr_c^2} P_1(r_c)$$

With that, both sides now match the expression from [1].

$$2(E_1 - E_2) P_1(r)P_2(r) = \frac{d}{dr} \left(\frac{dP_2(r)}{dr} P_1(r) - \frac{dP_1(r)}{dr} P_2(r) \right)$$

And finally, integrating both side of the expression [1] on the respective intervals, we obtain the exact expressions for [2] and [3].

Exercise 8

The exercise requires plotting the effective potential $V(r) = -\frac{Z}{r} + \frac{l(l+1)}{2r^2}$, for the orbitals s, p, d, f. Which corresponds to values of $l = 0, 1, 2, 3$, respectively. It is interesting to notice how the angular momentum affects the effective potential a particle experiences. Let's trace a rough, inaccurate, approximation of the max energy of an electron at each orbital, with a dashed line matching the color of the respective potential curve. For each of these electrons, the region defined by the intersections of the dashed line and the potential curve is the classical region the electron can exist. The higher the angular momentum, the farther away from the nucleus is the inner boundary of the region the electron can occupy, which can also translates in, the higher the angular momentum, the faster the potential grows closer to the nucleus.

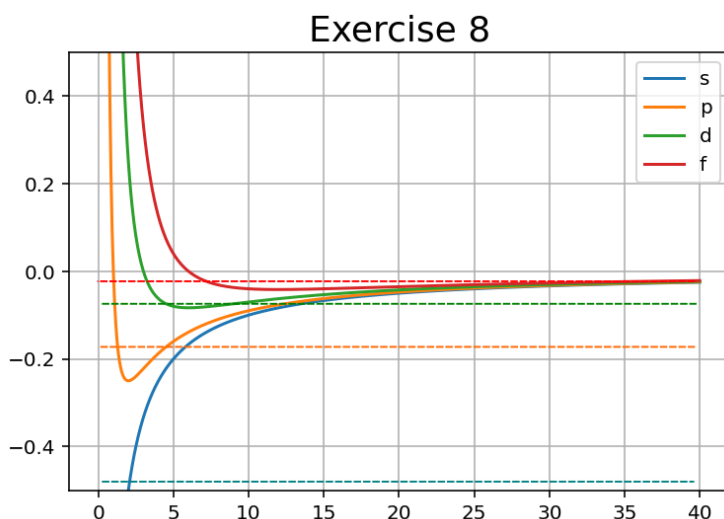


Figure 8: Plot for the effective Coulomb potential $V(r) = -\frac{Z}{r} + \frac{l(l+1)}{2r^2}$ for s, p, d, f, orbitals, $l = 0, 1, 2, 3$, respectively. The traced lines represent electrons at the given orbitals (matching colors) classically confined at those potential regions due to their angular momentum.

Exercise 9

On this exercise we use the code from *radial.py* to plot the radial functions for 1s, 2s, 3s, 4s, 6s, and 9s, as well as calculate the energy level at those states. The code implements Cowell-Numerov's method to obtain both the eigenvalue and eigenstate solving Schrödinger's equation numerically for 1-electron systems. On the Method session we elaborate further on how the algorithm works. For the purpose of this exercise, we will verify the number of nodes obtained from the program and the energy calculated, comparing it with the analytical formula $E = -\frac{Z}{2n^2}$.

From running the program, we obtained the following results for each of the input states:

Orbital	Nodes	Energy Eigenvalue (a.u.)	$(n - l - 1)$	$E = -Z/2n^2$
1s	0	-4.999×10^{-1}	0	-0.5
2s	1	-1.249×10^{-1}	1	-0.125
3s	2	-5.555×10^{-2}	2	-0.05555...
4s	3	-3.124×10^{-2}	3	-0.03125
6s	5	-1.388×10^{-2}	6	-0.01388...
9s	8	-6.172×10^{-3}	8	-0.0061728

Table 1: Energy eigenvalues and number of radial nodes for selected s orbitals.

From the table, we can verify the values obtained from the numerical method match the analytical result with satisfying accurate, for the purpose of this exercise.

All the plots are attached to the appendix session.

Exercise 10

This exercise requires deriving the logarithmic transformation applied to the radial Schrödinger's equation, use on *radiallog.py*.

This logarithmic transformation improves the algorithm performance and accuracy for as it better model the function behavior along the radius. As we discuss in the next exercise. Bellow is the original form of the radial Schrödinger's equation:

$$\frac{d^2 P(r)}{dr^2} + 2 \left(E - V(r) - \frac{l(l+1)}{2r^2} \right) P(r) = 0$$

We are given the following transformation:

$$\bar{P}(\rho) = \frac{P(r)}{\sqrt{r}}$$

$$r = \frac{e^\rho}{Z}$$

$$\frac{d\rho}{dr} = \frac{1}{Zr} Z = \frac{1}{r}$$

In order to apply the transformation, we must find the corresponding value for $\frac{d^2 P(r)}{dr^2}$. Let's start by the first-order derivative:

$$\frac{dP(r)}{dr} = \frac{r^{-\frac{1}{2}}}{2} \bar{P}(\rho) + \sqrt{r} \frac{d\bar{P}(\rho)}{d\rho} \frac{1}{r} = \frac{r^{-\frac{1}{2}}}{2} \bar{P}(\rho) + r^{-\frac{1}{2}} \frac{d\bar{P}(\rho)}{d\rho} = r^{-\frac{1}{2}} \left(\frac{\bar{P}(\rho)}{2} + \frac{d\bar{P}(\rho)}{d\rho} \right)$$

Using this result, we can now try to derive the second-order derivative:

$$\frac{d^2 P(r)}{dr^2} = -\frac{1}{2} r^{-\frac{3}{2}} \left(\frac{\bar{P}(\rho)}{2} + \frac{d\bar{P}(\rho)}{d\rho} \right) + r^{-\frac{1}{2}} \left(\frac{d\bar{P}(\rho)}{2d\rho} \frac{1}{r} + \frac{d^2 \bar{P}(\rho)}{d^2 \rho} \frac{1}{r} \right)$$

$$\begin{aligned}
&= -\frac{r^{-\frac{3}{2}}}{2} \left(\frac{\bar{P}(\rho)}{2} + \frac{d\bar{P}(\rho)}{d\rho} - \frac{d\bar{P}(\rho)}{d\rho} - \frac{2d^2\bar{P}(\rho)}{d^2\rho} \right) = -\frac{r^{-\frac{3}{2}}}{2} \left(\frac{\bar{P}(\rho)}{2} - \frac{2d^2\bar{P}(\rho)}{d^2\rho} \right) \\
&= -\frac{r^{-\frac{3}{2}}}{2} \left(\frac{\bar{P}(\rho)}{2} - 2\frac{d^2\bar{P}(\rho)}{d^2\rho} \right) + 2 \left(E - V(r) - \frac{l(l+1)}{2r^2} \right) \sqrt{r}P(\rho) = 0 \\
&= -\frac{r^{-\frac{3}{2}}}{2} \left(\frac{\bar{P}(\rho)}{2} - 2\frac{d^2\bar{P}(\rho)}{d^2\rho} \right) + \frac{2}{r^2} \left(r^2[E - V(r)] - \frac{l(l+1)}{2} \right) \sqrt{r}\bar{P}(\rho) \\
&= -\frac{r^{-\frac{3}{2}}}{2} \left(\frac{\bar{P}(\rho)}{2} - 2\frac{d^2\bar{P}(\rho)}{d^2\rho} \right) + \frac{r^{-\frac{3}{2}}}{2} (2r^2[E - V(r)] - l(l+1)) \bar{P}(\rho) \\
&= -\frac{\bar{P}(\rho)}{2} + 2\frac{d^2\bar{P}(\rho)}{d^2\rho} + (2r^2[E - V(r)] - l(l+1)) \bar{P}(\rho) \\
&= 2\frac{d^2\bar{P}(\rho)}{d^2\rho} + \left(2r^2[E - V(r)] - l^2 - l - \frac{1}{2} \right) \bar{P}(\rho) \\
&= 2\frac{d^2\bar{P}(\rho)}{d^2\rho} + 2 \left(r^2[E - V(r)] - l^2 - \frac{l}{2} - \frac{1}{4} \right) \bar{P}(\rho) \\
&\boxed{\frac{d^2\bar{P}(\rho)}{d^2\rho} + (r^2[E - V(r)] - (l + \frac{1}{2})^2) \bar{P}(\rho) = 0} \tag{4}
\end{aligned}$$

Exercise 12

This exercise asks to compare the accuracy and number of grid points use by both implementations of Cowell-Numerov method, *radial.py* and *radiallog.py*. Bellow, the 2 tables presents the associate state being calculated, time taken by each algorithm, number of grid points and energy value obtained:

Orbital	Time Used (s)	Grid Points	Energy Eigenvalue (a.u.)
1s	0.000794	657	$-5.0000000074843554 \times 10^{-1}$
2s	0.005504	690	$-1.2500000110858570 \times 10^{-1}$
3s	0.006413	710	$-5.5555557384279856 \times 10^{-2}$
4s	0.009248	724	$-3.1250002853761609 \times 10^{-2}$
6s	0.010820	743	$-1.3888894684640048 \times 10^{-2}$
9s	0.016067	763	$-6.1728519350589418 \times 10^{-3}$

Table 2: Output values from *radiallog.py*.

It is clear from the data the efficiency and accuracy of *radiallog.py* is much superior to *radial.py*. The number of grid points use by *radiallog.py* is at least an order of magnitude lower, although dynamically calculated, and the accuracy (if the analytical solution is taken

Orbital	Time Used (s)	Grid Points	Energy Eigenvalue (a.u.)
1s	0.014377	10000	$-4.9999468702903238 \times 10^{-1}$
2s	0.063277	10000	$-1.2499735429393080 \times 10^{-1}$
3s	0.093079	10000	$-5.5553798964794107 \times 10^{-2}$
4s	0.134526	10000	$-3.1248687980098656 \times 10^{-2}$
6s	0.152343	10000	$-1.3888021453930760 \times 10^{-2}$
9s	0.196393	10000	$-6.1722684629813181 \times 10^{-3}$

Table 3: Output values from `radial.py`.

as parameter) is superior. With less grid points to be calculated, naturally the execution is much faster.

The difference in strategy is based on the adoption of a denser grid near the origin and more spaced further away. Intuitively, this reasoning can be backed by how the potential curve behaves closer to the nucleus, as observed in exercise 8. The potential magnitude grows exponentially the closer to the nucleus, for which smaller intervals would be necessary to better interpolate the curve, and a logarithm grid would naturally fit as it is the inverse of the exponential function. And in compensation, the grid further away, where the potential growth drops very fast, the intervals became sparse, which contributes for the better performance of the strategy as fewer points need to be calculated.

Exercise 13

The exercise requires plotting the radial wave function for 1s, 2s, 3s, 4s, 6s and 9s, using *radiallog.py* with some alterations, so all curves are plot together.

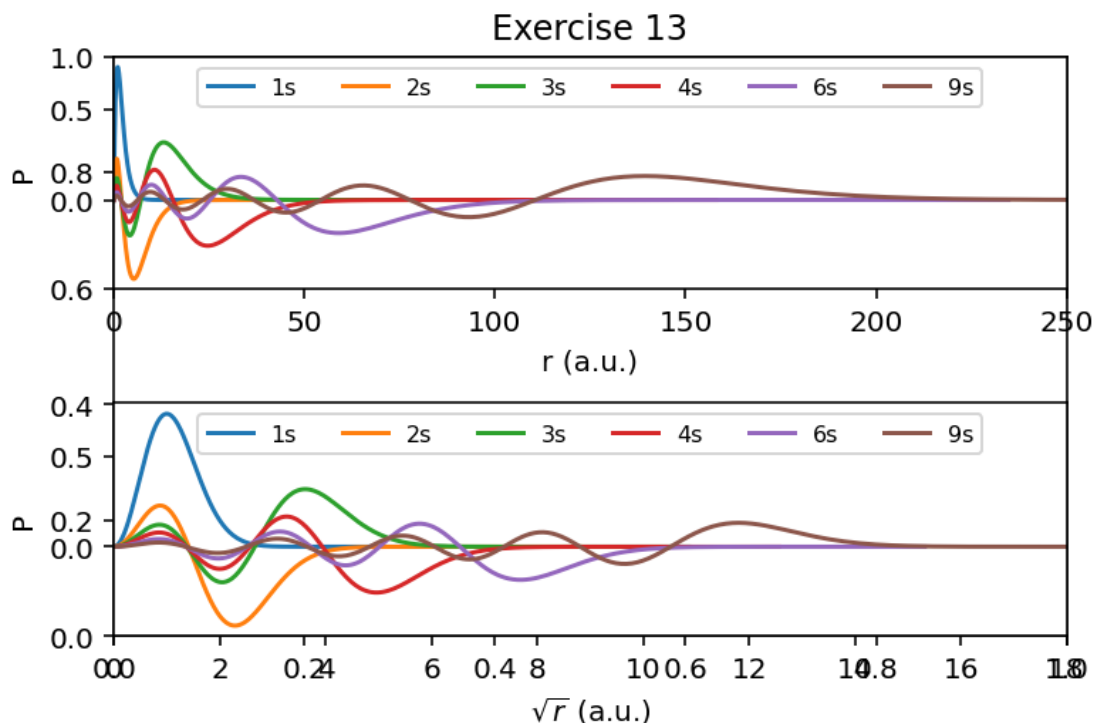


Figure 9: Plot of orbitals 1s, 2s, 3s, 4s, 6s, 9s, obtained from *radiallog.py*, showing the radial wave function along the axis r for the radius distance from the nucleus.

To obtain the plot from Figure 9, the following alterations were made to the main function on *radiallog.py*:

```

1 if __name__ == '__main__':
2     plt.figure()
3     plt.title("Exercise 13")
4
5     result={}
6     for n in [1,2,3,4,6,9]:
7         r,P=radiallog(0,n,1,False,)
8         plt.subplot(211)
9         plt.xlim(0, 250)
10        label = format("%ds" %(n))
11        plt.plot(r, np.sqrt(r) * P, label=label)
12        plt.xlabel("r (a.u.)")
13        plt.ylabel("P")
14        plt.legend(loc="upper center", ncols=6, fontsize=8)
15

```

```

16     rr = np.sqrt(r)
17
18     plt.subplot(212)
19     plt.plot(rr, np.sqrt(r) * P, label=label)
20     plt.xlabel('$\sqrt{\mathrm{r}}$ (a.u.)')
21     plt.ylabel("P")
22     plt.xlim(0, 18)
23     plt.legend(loc="upper center", ncols=6, fontsize=8)
24 plt.legend(loc="upper center", ncols=6, fontsize=8)
25 plt.tight_layout(pad=1.0)
26 plt.xlabel(r"$\sqrt{r}$ (a.u.)")
27 plt.ylabel("P")
28 plt.show()

```

Exercise 14

This exercise requires using *radiallog.py* to plot 2p, 3p, 4p, 5p, 7p and 9p, orbitals. We are asked to compare the number of nodes and energy values obtained with the analytical formulas. Bellow we correlate each state/orbital with the respective value obtained from the analytical solutions:

Orbital	Nodes	Energy Eigenvalue (a.u.)	$(n - l - 1)$	$E = -Z/2n^2$
2p	0	-1.250×10^{-1}	0	-0.125
3p	1	-5.555×10^{-2}	1	-0.05555...
4p	2	-3.125×10^{-2}	2	-0.03125
5p	3	-2.000×10^{-2}	3	-0.02
7s	5	-1.020×10^{-2}	5	-0.01020
9s	7	-6.172×10^{-3}	7	-0.006172

Table 4: Energy eigenvalues and number of radial nodes for selected *s* orbitals.

We can observe the values obtained from the program match the expectation from the analytical solution with a satisfactory degree of precision. Bellow we can verify the plot for all states combined.

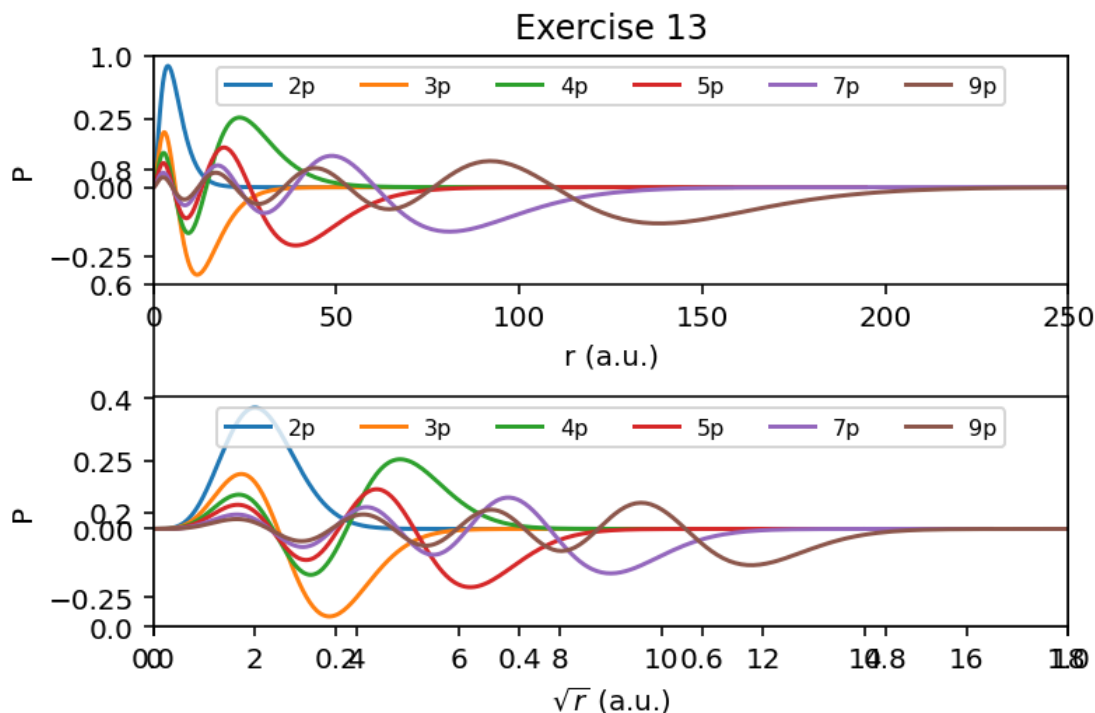


Figure 10: Plot of orbitals 2p, 3p, 4p, 5p, 7s and 9s, obtained from *radiolog.py*, showing the radial wave function along the axis r for the radius distance from the nucleus.

Exercise 15

The orbital quantum number relates to the angular momentum of the particle. From Schrodinger's radial equation, it corresponds to a factor of $\frac{l(l+1)}{2r^2}$, added to the effective potential. Mathematically, this cancels out the central potential factor, which is more intense near the nucleus. Below is a comparison of the "growth" rate of both central potential and angular momentum related potential relative to the distance from the nucleus. Other factors such as Z and l^2 , will change how the factors balance each other, but it is easy to see, how near the nucleus they follow the same trend, canceling each other, and the larger the value of l , the slower would be the decline of the angular momentum curve, as $U_l(r) \propto l^2$.

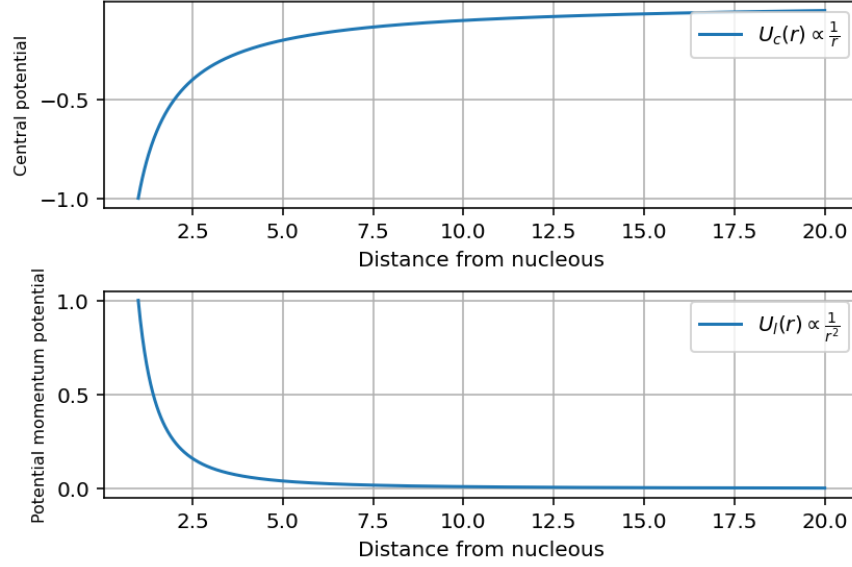


Figure 11: Comparison of growth ratio for central potential and angular momentum related potential.

On the orbital s , the quantum number $l = 0$, so no angular momentum related potential balances the central potential, which then completely dictates how the effective potential curve behaves. With a steep decline of intensity, parting from the nucleus.

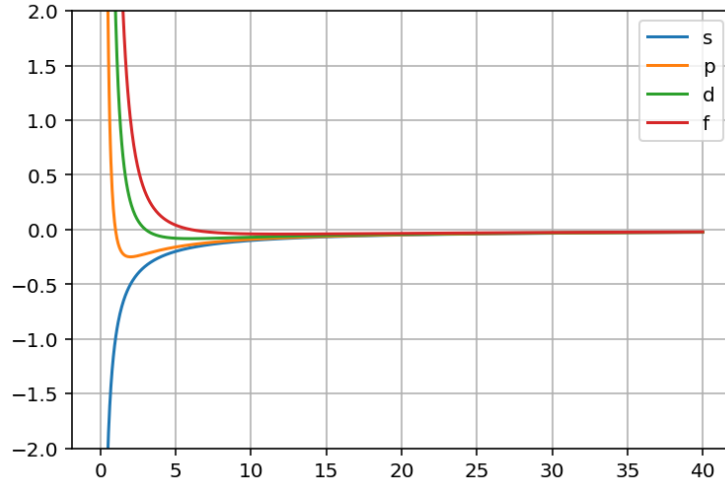


Figure 12: Plot for the effective Coulomb potential $V(r) = -\frac{Z}{r} + \frac{l(l+1)}{2r^2}$ for s , p , d , f , orbitals, $l = 0, 1, 2, 3$, respectively.

With a higher potential intensity closer to the nucleus, the system's energy at this region is concentrated as potential energy, with the kinetic energy of the particle then much lower, and consequently, its speed being much lower, the density of radial distances the particle

can occupy on this region is much higher, or to make a classical analogy, the particle spend much more time on this region, being slower there.

Exercise 16

The exercise provides the following expression for the radial expectation:

$$\langle r \rangle = \int_0^\infty r P_{nl}(r) dr$$

This expression will calculate the mean value of the radius for a given radial function $P_{nl}(r)$. Meaning, the most likely location along the radius in which particles belonging to the state expressed by the radial function, can be found. Perhaps, interesting to note, this region do not concentrate exactly around where the wave-function peak. It is instead, for the orbitals analyzed (see appendix), slightly shifted to the right (in the positive direction). This happens due to the small, but still existing probabilities of finding the particle beyond the orbital region concentration, to $r \rightarrow \infty$.

Exercise 17

The exercise requires integrating the radial function, obtained using Cowell-Numerov, along r up until the practical infinity. This can be achieved in a simple way using the following code snippet:

```

1  # The following code uses a simple trapezio approximation to integrate
   the
2  # radial function along the radius from 0 to the practical infinity.
3  r,P,E =radiallog(0, 1, 1,False)
4  PIntegral = 0
5  for i in range(1, len(P)):
6      PIntegral = PIntegral + (r[i] - r[i-1]) * (P[i] + P[i-1])/2
7
8  print("Integration of the radial function up to infinity: %f" %
      PIntegral)

```

Output:

```

1 Integration of the radial function up to infinity: 1.772582

```

Exercise 18 and 19

This exercise requires modifying *radiallog.py* to use the parametric potential, with $\alpha = 0.2683$. The expression for the parametric potential given by:

$$V(r) = -\frac{Z}{r} + \frac{(Z - \zeta)r}{\alpha^2 + r^2}$$

With the modification, the program can approximate the many-electrons problem by accounting for the potential contribution of the electrons repelling each other. With electrons

closer to the nucleus subject to a potential closer to the Coulomb's potential $-\frac{Z}{r}$, and electrons further away, with $r \rightarrow \infty$, more strongly affected by the $\zeta = Z - N + 1$ electrons contribution.

The following results were obtained for the different orbitals, matching the values presented on Figure 7, section 15 of the project description [JE22].

Table 5: Calculated one-electron orbital energies for Na I.

Orbital	Nodes	Energy (a.u.)
$1s$	0	-47.46537142
$2s$	1	-1.64520694
$2p$	0	-0.55320827
$3s$	2	-0.19194458
$3p$	1	-0.10722888
$3d$	0	-0.05751429
$4s$	3	-0.07203737
$4p$	2	-0.04943445
$4d$	1	-0.03207466
$4f$	0	-0.03152388
$5s$	4	-0.03771008

From the results above, we calculated the following ΔE values, for the excited states of NaI , again, matching the values provided on section 15 of the project description, converted from $a.u.$ to cm^{-1} by the given factor of 219474.63, from the project description, section 15.

Table 6: Calculated Na I state energy differences relative to the 3s base state.

State	Configuration	ΔE (cm^{-1})
Base state	$1s^2 2s^2 2p^6 3s$	0.00
3p	$1s^2 2s^2 2p^6 3p$	18592.95
4s	$1s^2 2s^2 2p^6 4s$	26316.59
3d	$1s^2 2s^2 2p^6 3d$	29504.04
4p	$1s^2 2s^2 2p^6 4p$	31277.36
5s	$1s^2 2s^2 2p^6 5s$	33850.56
4d	$1s^2 2s^2 2p^6 4d$	35087.39
4f	$1s^2 2s^2 2p^6 4f$	35208.27

Orbital plots for Sodium (Na I)

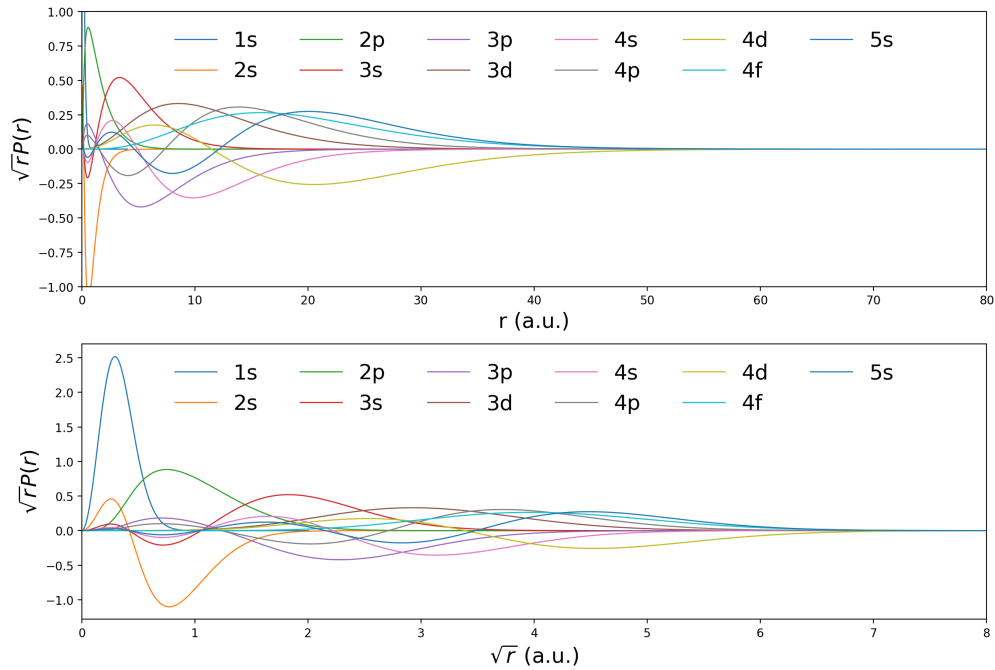


Figure 13: Plot obtained from a modified version of *radiallog.py*, for Sodium Na I, $Z = 11$, $N = 11$.

Exercise 20

Similar to the previous exercise, we are required to modify *radiallog.py* to calculate the ΔE , comparatively with the base state, of excited states for Al III, $Z=13$. However, we are expected to adjust the value of α , for the parametric potential, so the values obtained approximate the experimental reference from NIST [?].

After a few tests, the value which relatively best approximate the reference was of $\alpha = 0.2073$. For which the following energy values where obtained:

Table 7: Calculated one-electron orbital energies for Al III.

Orbital	Nodes	Energy (a.u.)
1s	0	-66.85146675
2s	1	-4.41712856
2p	0	-2.76440092
3s	2	-1.06038756
3p	1	-0.79100914
3d	0	-0.53346941
4s	3	-0.47538560
4p	2	-0.38797257
4d	1	-0.29520701
4f	0	-0.28573017
5s	4	-0.26990207

For the base and excited states, the following table relates the values calculated.

Table 8: Calculated Al III state energy differences relative to the 3s base state.

State	Configuration	ΔE (cm ⁻¹)
Base state	$2p^6 3s^1$	0.00
$3p$	$2p^6 3p^1$	59121.73
$3d$	$2p^6 3d^1$	115645.17
$4s$	$2p^6 4s^1$	128393.09
$4p$	$2p^6 4p^1$	147578.03
$4d$	$2p^6 4d^1$	167937.72
$4f$	$2p^6 4f^1$	170017.64
$5s$	$2p^6 5s^1$	173491.51

We can compare with the reference values from NIST and verify the approximations follow a similar trend as what was obtained for Na I, with $\alpha = 0.2683$.

Table 9: Experimental energy levels for Al III from NIST.

Configuration	Term	J	Level (cm^{-1})
$2p^6 3s$	2S	1/2	0.00
$2p^6 3p$	$^2P^\circ$	1/2	53682.93
		3/2	53916.60
$2p^6 3d$	2D	5/2	115956.21
		3/2	115958.50
$2p^6 4s$	2S	1/2	126164.05
$2p^6 4p$	$^2P^\circ$	1/2	143633.38
		3/2	143713.50
$2p^6 4d$	2D	5/2	165786.32
		3/2	165787.51
$2p^6 4f$	$^2F^\circ$	5/2	167612.75
		7/2	167613.37
$2p^6 5s$	2S	1/2	170637.35
$2p^6 5p$	$^2P^\circ$	1/2	178433.43
		3/2	178470.32

And in regard to the order in which the orbitals are presented, it is due to the energy gap between the electron on the base state and how much energy is necessary to take it to the respective degenerate/excited state.

Orbital plots for Sodium (Al III)

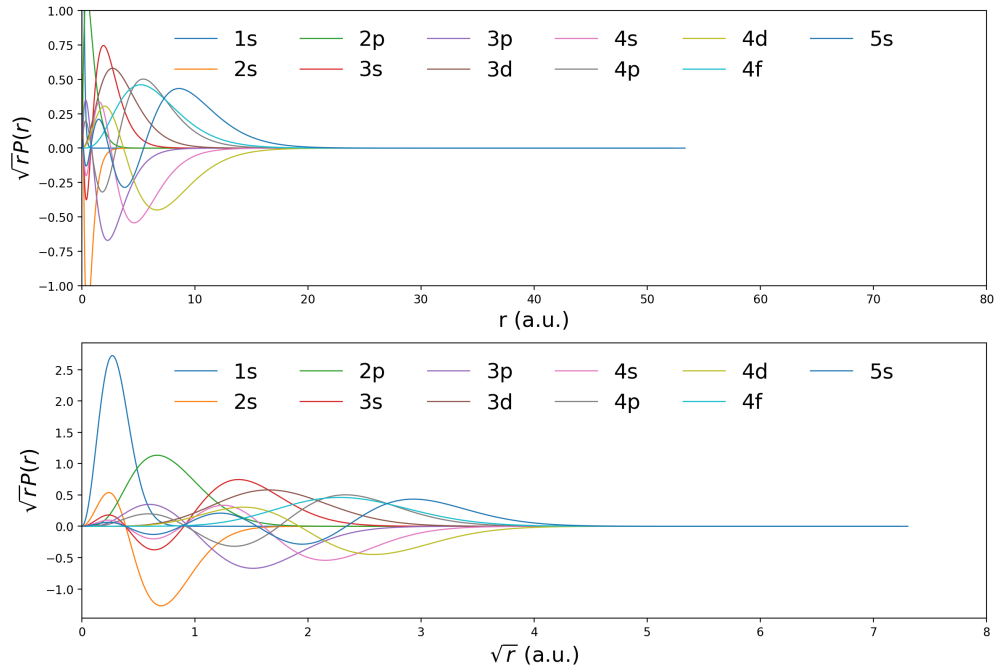


Figure 14: Plot obtained from a modified version of *radiallog.py*, for Al III, $Z = 13$, $N = 11$.

Exercise 21

On this exercise, we are now asked to calculate the ΔE of excited states for Li I. However, it poses the question if the process to find α , for the parametric potential, can be automated. And the answer is yes.

The function *FindAlphaParameter* was written for this purpose. The algorithm performs a task in someways, analogous to Cowell-Numerov algorithm, in the sense that we make an initial guess for α , and we calculate the energy values based on this alpha, incrementing or decrementing alpha at each step, based on the error obtained from the previous iteration. The error is calculated as the average difference of the reference values for ΔE obtained from NIST and the value calculated using our α . The increment or decrement of α is done by $-\frac{\text{error}}{\text{mean}(\Delta E_{\text{NIST}})}$. When the error is less than an arbitrary tolerance value, the algorithm stops. The full code can be found on the appendix session, together with the settings for running with the elements from the previous exercises. Bellow is a snippet for the algorithm main loop:

```
1  while(abs(error) > 100):
2      # Adjust the parameter in the contrary direction to the error,
      relative to the mean of the reference values
3      alpha = alpha + alpha*(-error)/abs(meanERef)
4
5      # Reset values for new round
6      levels = []
7      alphaLast = alpha
8
9      for (n, l, dltaE) in list(Eref.values()):
10         E = radiallog(n, l, N, Z, alpha, False)[2]
11         E = AUnitToSpectroscopic(E)
12         levels.append(E)
13         meanDeltE = (np.array(levels) - levels[0]).mean()
14
15         # If we obtained some improvement on this round, we update the
      parameter and try again
16         error = meanDeltE - meanERef
```

Table 10: Calculated one-electron orbital energies.

Orbital	Energy (a.u.)
$1s$	-1.70397098
$2s$	-0.19815123
$2p$	-0.13221359
$3s$	-0.07417128
$3p$	-0.05763785
$3d$	-0.05595781
$4s$	-0.03860571
$4p$	-0.03211894
$4d$	-0.03141940
$4f$	-0.03130994
$5s$	-0.02363088

Table 11: Calculated energy differences relative to the $1s^2 2s$ base state.

State	Configuration	ΔE (cm $^{-1}$)
Base state	$1s^2 2s^1$	0.00
$2p$	$1s^2 2p^1$	14471.64
$3s$	$1s^2 3s^1$	27210.45
$3p$	$1s^2 3p^1$	30839.12
$3d$	$1s^2 3d^1$	31207.85
$4s$	$1s^2 4s^1$	35016.19
$4p$	$1s^2 4p^1$	36439.88
$4d$	$1s^2 4d^1$	36593.41

Once again, the values we found have a good approximation in comparison to the experimental values from NIST.

Table 12: Experimental energy levels from NIST.

Configuration	Term	J	Level (cm^{-1})
$1s^2 2s$	2S	1/2	0.00
$1s^2 2p$	$^2P^\circ$	1/2	14903.66
		3/2	14904.00
$1s^2 3s$	2S	1/2	27206.12
$1s^2 3p$	$^2P^\circ$	1/2	30925.38
		3/2	30925.38
$1s^2 3d$	2D	3/2	31283.08
		5/2	31283.12
$1s^2 4s$	2S	1/2	35012.06
$1s^2 4p$	$^2P^\circ$	1/2	36469.55
		3/2	36469.55
$1s^2 4d$	2D	3/2	36623.38
		5/2	36623.40
$1s^2 4f$	$^2F^\circ$	5/2	36630.2
		7/2	36630.2

Orbital plots

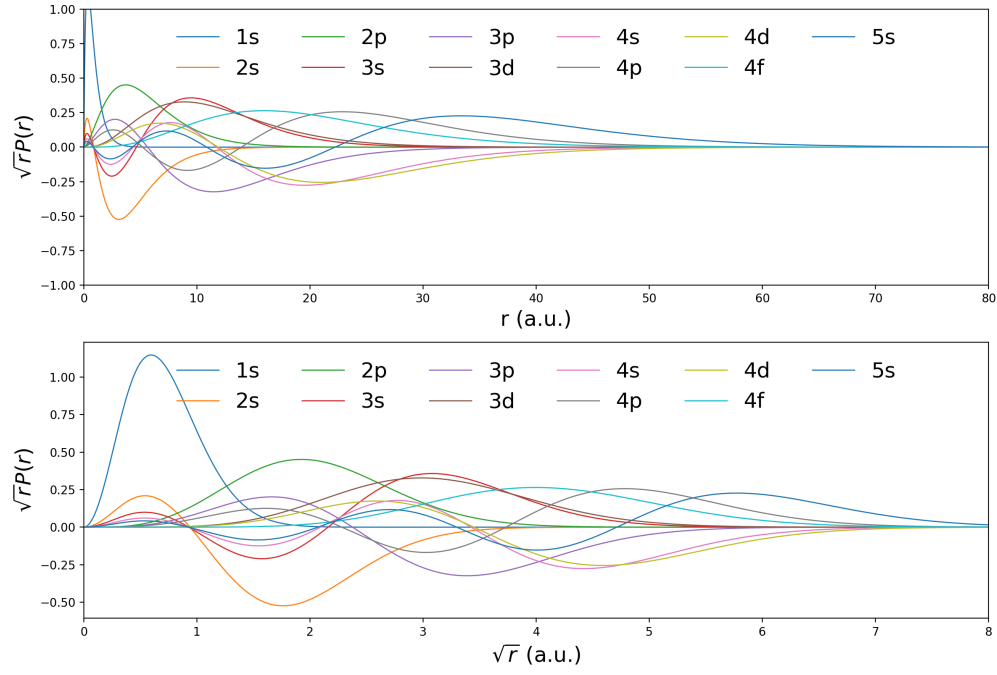


Figure 15: Plot obtained from a modified version of *radiallog.py*, for Li I, $Z = 3$, $N = 3$.

A Exercise 5 - Spherical Harmonics Orthogonality check

Code

```
1 import sympy as sym
2 import matplotlib.pyplot as plt
3
4 #
5 =====
6
7 # Using SymPy functions to perform the integrations, verify that the
8   spheri-
9 # cal harmonics given on page 2 satisfy the following orthogonality
10  relations
11 #
12 =====
13
14
15 theta, phi = sym.symbols("\\theta \\phi")
16
17 Y00 = sym.Function('Y_{00}')(theta, phi)
18 Y00_expr = lambda m_sign: 1/sym.sqrt(4*sym.pi)
19
20 Y11_plus = sym.Function('Y_{1+1}')(theta, phi)
21 Y11_minus = sym.Function('Y_{1-1}')(theta, phi)
22 Y11_expr = lambda m_sign: -m_sign * sym.sqrt(sym.Rational(3/8)/sym.pi) *
23   sym.sin(theta) * sym.exp( m_sign * sym.I*phi)
24
25 Y10 = sym.Function('Y_{10}')(theta, phi)
26 Y10_expr = lambda m_sign: sym.sqrt(sym.Rational(3/4)/sym.pi) * sym.cos(
27   theta) * sym.exp(2*sym.I*phi)
28
29 Y22_plus = sym.Function('Y_{2+2}')(theta, phi)
30 Y22_minus = sym.Function('Y_{2-2}')(theta, phi)
31 Y22_expr = lambda m_sign: sym.sqrt(sym.Rational(15/32)/sym.pi) * sym.sin(
32   theta)**2 * sym.exp(m_sign * 2 * sym.I*phi)
33
34 Y21_plus = sym.Function('Y_{2+1}')(theta, phi)
35 Y21_minus = sym.Function('Y_{2-1}')(theta, phi)
36 Y21_expr = lambda m_sign: -m_sign*sym.sqrt(sym.Rational(15/8)/sym.pi) *
37   sym.sin(theta)*sym.cos(theta) * sym.exp(m_sign * sym.I*phi)
38
39 Y20 = sym.Function('Y_{20}')(theta, phi)
40 Y20_expr = lambda m_sign: sym.sqrt(sym.Rational(5/16)/sym.pi) * (2*sym.cos
41   (theta)**2 - sym.sin(theta)**2)
42
43
44 # Now we try calculating the inner product between the functions
45 func_sym = [Y00, Y11_plus, Y11_minus, Y10, Y22_plus, Y22_minus, Y21_plus,
46   Y21_minus, Y20]
```

```

36 func_expr = [Y00_expr(1), Y11_expr(1), Y11_expr(-1), Y10_expr(1), Y22_expr
    (1), Y22_expr(-1), Y21_expr(1), Y21_expr(-1), Y20_expr(1)]
37
38
39
40 for i in range(0, len(func_sym)):
41
42     rows = len(func_sym) - i
43     fig_height = 1 + 0.5 * rows
44
45     fig, ax = plt.subplots(figsize=(14, fig_height))
46     ax.axis("off")
47
48     title = (
49         rf"${sym.latex(func_sym[i])}$ Orthogonality check against:"
50         "\n"
51         rf"${sym.latex(func_sym[i:])}$"
52     )
53
54     ax.text(0.5, 1, title, ha="center", va="top", transform=ax.transAxes,
        fontsize=14)
55
56     # Body text
57     y = 0.8 - i/(2*len(func_sym)) # try accounting for the image
        distortion with less rows
58     dy = 0.75 / max(rows, 1)
59
60     for j in range(i, len(func_sym)):
61         # Calculate the double integral for the functional inner product
62         integrand = sym.conjugate(func_expr[i]) * func_expr[j] * sym.sin(
            theta)
63         integral = sym.Integral(integrand, (theta, 0, sym.pi), (phi, 0, 2*
            sym.pi))
64
65         # Evaluate the expression
66         integral_value = sym.integrate(integrand, (theta, 0, sym.pi), (phi
            , 0, 2*sym.pi))
67
68         # Obtain the inner product expression
69         inner_product = format(r"\langle %s, %s \rangle" % (sym.latex(
            func_sym[i]), sym.latex(func_sym[j])))
70         # Obtain the integral expression in latex
71         integral_latex = sym.latex(integral)
72         # Replace the latex expression \int_limits_{ }^{ } for the supported
            \int_{ }^{ }
73         integral_latex = integral_latex.replace("\limits_", "_")
74
75         print(integral_latex)
76         print(integral_value)
77         print(inner_product)
78

```

```

79     line = rf"${inner_product} = {integral_latex} = {sym.latex(
integral_value)}$"
80
81     ax.text(
82         0.05, y,
83         line,
84         ha="left",
85         va="top",
86         transform=ax.transAxes,
87         fontsize=13
88     )
89     y -= dy
90
91 plt.show()

```


Results

$Y_{00}(\theta, \phi)$ Orthogonality check against:

$$[Y_{00}(\theta, \phi), Y_{1+1}(\theta, \phi), Y_{1-1}(\theta, \phi), Y_{10}(\theta, \phi), Y_{2+2}(\theta, \phi), Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$$

$$\begin{aligned} \langle Y_{00}(\theta, \phi), Y_{00}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\sin(\theta)}{4\pi} d\theta d\phi = 1 \\ \langle Y_{00}(\theta, \phi), Y_{1+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{\sqrt{6} e^{i\phi} \sin^2(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{1-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{6} e^{-i\phi} \sin^2(\theta)}{8\pi} d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{10}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{3} e^{2i\phi} \sin(\theta) \cos(\theta)}{4\pi} d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2+2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{30} e^{2i\phi} \sin^3(\theta)}{16\pi} d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2-2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{30} e^{-2i\phi} \sin^3(\theta)}{16\pi} d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{\sqrt{30} e^{i\phi} \sin^2(\theta) \cos(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{30} e^{-i\phi} \sin^2(\theta) \cos(\theta)}{8\pi} d\theta d\phi = 0 \\ \langle Y_{00}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{5} (-\sin^2(\theta) + 2\cos^2(\theta)) \sin(\theta)}{8\pi} d\theta d\phi = 0 \end{aligned}$$

Figure 16: ...

$Y_{1+1}(\theta, \phi)$ Orthogonality check against:

$$[Y_{1+1}(\theta, \phi), Y_{1-1}(\theta, \phi), Y_{10}(\theta, \phi), Y_{2+2}(\theta, \phi), Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$$

$$\begin{aligned} \langle Y_{1+1}(\theta, \phi), Y_{1+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 e^{i\phi} e^{-i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta})}{8\pi} d\theta d\phi = 1 \\ \langle Y_{1+1}(\theta, \phi), Y_{1-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{3 e^{-i\bar{\phi}} e^{-i\phi} \sin^2(\theta) \sin(\bar{\theta})}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{1+1}(\theta, \phi), Y_{10}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{3\sqrt{2} e^{2i\phi} e^{-i\bar{\phi}} \sin(\theta) \sin(\bar{\theta}) \cos(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{1+1}(\theta, \phi), Y_{2+2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{3\sqrt{5} e^{2i\phi} e^{-i\bar{\phi}} \sin^3(\theta) \sin(\bar{\theta})}{16\pi} \right) d\theta d\phi = 0 \\ \langle Y_{1+1}(\theta, \phi), Y_{2-2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{3\sqrt{5} e^{-2i\phi} e^{-i\bar{\phi}} \sin^3(\theta) \sin(\bar{\theta})}{16\pi} \right) d\theta d\phi = 0 \\ \langle Y_{1+1}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3\sqrt{5} e^{i\phi} e^{-i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta}) \cos(\theta)}{8\pi} d\theta d\phi = 0 \\ \langle Y_{1+1}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{3\sqrt{5} e^{-i\phi} e^{-i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta}) \cos(\theta)}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{1+1}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{\sqrt{30} (-\sin^2(\theta) + 2\cos^2(\theta)) e^{-i\bar{\phi}} \sin(\theta) \sin(\bar{\theta})}{16\pi} \right) d\theta d\phi = 0 \end{aligned}$$

Figure 17: ...

$Y_{1-1}(\theta, \phi)$ Orthogonality check against:
 $[Y_{1-1}(\theta, \phi), Y_{10}(\theta, \phi), Y_{2+2}(\theta, \phi), Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$

$$\begin{aligned}
\langle Y_{1-1}(\theta, \phi), Y_{1-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 e^{-i\phi} e^{i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta})}{8\pi} d\theta d\phi = 1 \\
\langle Y_{1-1}(\theta, \phi), Y_{10}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 \sqrt{2} e^{2i\phi} e^{i\bar{\phi}} \sin(\theta) \sin(\bar{\theta}) \cos(\theta)}{8\pi} d\theta d\phi = 0 \\
\langle Y_{1-1}(\theta, \phi), Y_{2+2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 \sqrt{5} e^{2i\phi} e^{i\bar{\phi}} \sin^3(\theta) \sin(\bar{\theta})}{16\pi} d\theta d\phi = 0 \\
\langle Y_{1-1}(\theta, \phi), Y_{2-2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 \sqrt{5} e^{-2i\phi} e^{i\bar{\phi}} \sin^3(\theta) \sin(\bar{\theta})}{16\pi} d\theta d\phi = 0 \\
\langle Y_{1-1}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{3\sqrt{5} e^{i\phi} e^{i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta}) \cos(\theta)}{8\pi} \right) d\theta d\phi = 0 \\
\langle Y_{1-1}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 \sqrt{5} e^{-i\phi} e^{i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta}) \cos(\theta)}{8\pi} d\theta d\phi = 0 \\
\langle Y_{1-1}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{30} (-\sin^2(\theta) + 2\cos^2(\theta)) e^{i\bar{\phi}} \sin(\theta) \sin(\bar{\theta})}{16\pi} d\theta d\phi = 0
\end{aligned}$$

Figure 18: ...

$Y_{10}(\theta, \phi)$ Orthogonality check against:
 $[Y_{10}(\theta, \phi), Y_{2+2}(\theta, \phi), Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$

$$\begin{aligned}
\langle Y_{10}(\theta, \phi), Y_{10}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 e^{2i\phi} e^{-2i\bar{\phi}} \sin(\theta) \cos(\theta) \cos(\bar{\theta})}{4\pi} d\theta d\phi = 1 \\
\langle Y_{10}(\theta, \phi), Y_{2+2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 \sqrt{10} e^{2i\phi} e^{-2i\bar{\phi}} \sin^3(\theta) \cos(\bar{\theta})}{16\pi} d\theta d\phi = 0 \\
\langle Y_{10}(\theta, \phi), Y_{2-2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 \sqrt{10} e^{-2i\phi} e^{-2i\bar{\phi}} \sin^3(\theta) \cos(\bar{\theta})}{16\pi} d\theta d\phi = 0 \\
\langle Y_{10}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{3\sqrt{10} e^{i\phi} e^{-2i\bar{\phi}} \sin^2(\theta) \cos(\theta) \cos(\bar{\theta})}{8\pi} \right) d\theta d\phi = 0 \\
\langle Y_{10}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 3 \sqrt{10} e^{-i\phi} e^{-2i\bar{\phi}} \sin^2(\theta) \cos(\theta) \cos(\bar{\theta})}{8\pi} d\theta d\phi = 0 \\
\langle Y_{10}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi \sqrt{15} (-\sin^2(\theta) + 2\cos^2(\theta)) e^{-2i\bar{\phi}} \sin(\theta) \cos(\bar{\theta})}{8\pi} d\theta d\phi = 0
\end{aligned}$$

Figure 19: ...

$Y_{2+2}(\theta, \phi)$ Orthogonality check against:
 $[Y_{2+2}(\theta, \phi), Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$

$$\begin{aligned}
\langle Y_{2+2}(\theta, \phi), Y_{2+2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 15 e^{2i\phi} e^{-2i\bar{\phi}} \sin^3(\theta) \sin^2(\bar{\theta})}{32\pi} d\theta d\phi = 1 \\
\langle Y_{2+2}(\theta, \phi), Y_{2-2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 15 e^{-2i\phi} e^{-2i\bar{\phi}} \sin^3(\theta) \sin^2(\bar{\theta})}{32\pi} d\theta d\phi = 0 \\
\langle Y_{2+2}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{15 e^{i\phi} e^{-2i\bar{\phi}} \sin^2(\theta) \sin^2(\bar{\theta}) \cos(\theta)}{16\pi} \right) d\theta d\phi = 0 \\
\langle Y_{2+2}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 15 e^{-i\phi} e^{-2i\bar{\phi}} \sin^2(\theta) \sin^2(\bar{\theta}) \cos(\theta)}{16\pi} d\theta d\phi = 0 \\
\langle Y_{2+2}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 5 \sqrt{6} (-\sin^2(\theta) + 2\cos^2(\theta)) e^{-2i\bar{\phi}} \sin(\theta) \sin^2(\bar{\theta})}{32\pi} d\theta d\phi = 0
\end{aligned}$$

Figure 20: ...

$Y_{2-2}(\theta, \phi)$ Orthogonality check against:
 $[Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$

$$\begin{aligned}\langle Y_{2-2}(\theta, \phi), Y_{2-2}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 15 e^{-2i\phi} e^{2i\bar{\phi}} \sin^3(\theta) \sin^2(\bar{\theta})}{32\pi} d\theta d\phi = 1 \\ \langle Y_{2-2}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{15 e^{i\phi} e^{2i\bar{\phi}} \sin^2(\theta) \sin^2(\bar{\theta}) \cos(\theta)}{16\pi} \right) d\theta d\phi = 0 \\ \langle Y_{2-2}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 15 e^{-i\phi} e^{2i\bar{\phi}} \sin^2(\theta) \sin^2(\bar{\theta}) \cos(\theta)}{16\pi} d\theta d\phi = 0 \\ \langle Y_{2-2}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 5 \sqrt{6} (-\sin^2(\theta) + 2\cos^2(\theta)) e^{2i\bar{\phi}} \sin(\theta) \sin^2(\bar{\theta})}{32\pi} d\theta d\phi = 0\end{aligned}$$

Figure 21: ...

$Y_{2+1}(\theta, \phi)$ Orthogonality check against:
 $[Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$

$$\begin{aligned}\langle Y_{2+1}(\theta, \phi), Y_{2+1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 15 e^{i\phi} e^{-i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta}) \cos(\theta) \cos(\bar{\theta})}{8\pi} d\theta d\phi = 1 \\ \langle Y_{2+1}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{15 e^{-i\phi} e^{-i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta}) \cos(\theta) \cos(\bar{\theta})}{8\pi} \right) d\theta d\phi = 0 \\ \langle Y_{2+1}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \left(-\frac{5 \sqrt{6} (-\sin^2(\theta) + 2\cos^2(\theta)) e^{-i\bar{\phi}} \sin(\theta) \sin(\bar{\theta}) \cos(\bar{\theta})}{16\pi} \right) d\theta d\phi = 0\end{aligned}$$

Figure 22: ...

$Y_{2-1}(\theta, \phi)$ Orthogonality check against:
 $[Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi)]$

$$\begin{aligned}\langle Y_{2-1}(\theta, \phi), Y_{2-1}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 15 e^{-i\phi} e^{i\bar{\phi}} \sin^2(\theta) \sin(\bar{\theta}) \cos(\theta) \cos(\bar{\theta})}{8\pi} d\theta d\phi = 1 \\ \langle Y_{2-1}(\theta, \phi), Y_{20}(\theta, \phi) \rangle &= \int_0^{2\pi} \int_0^\pi \frac{\pi 5 \sqrt{6} (-\sin^2(\theta) + 2\cos^2(\theta)) e^{i\bar{\phi}} \sin(\theta) \sin(\bar{\theta}) \cos(\bar{\theta})}{16\pi} d\theta d\phi = 0\end{aligned}$$

Figure 23: ...

$Y_{20}(\theta, \phi)$ Orthogonality check against:
 $[Y_{20}(\theta, \phi)]$

$$\langle Y_{20}(\theta, \phi), Y_{20}(\theta, \phi) \rangle = \int_0^{2\pi} \int_0^\pi \frac{\pi 5 (-\sin^2(\theta) + 2\cos^2(\theta)) (-\sin^2(\bar{\theta}) + 2\cos^2(\bar{\theta})) \sin(\theta)}{16\pi} d\theta d\phi = 1$$

Figure 24: ...

B Exercise 8 - Potential regions code

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 #
6 # =====
7 # Set Z = 1 and plot the effective potential  $V(r) = -Z/r + l(l+1)/2r^2$ 
8 # for s, p, d, and f electrons (see figure 4). Plot all the potentials in
9 #
10 # =====
11
12 Z = 1
13 def V(r, l):
14     return -Z/r + (l*(l+1))/(2*(r**2))
15
16 fig, ax = plt.subplots()
17 ax.set_title("Exercise 8", fontsize=18)
18
19 x = np.linspace(0.1, 40, 1000)
20
21 ax.plot(x, V(x, 0), label=r"s")
22 ax.plot(x, V(x, 1), label=r"p")
23 ax.plot(x, V(x, 2), label=r"d")
24 ax.plot(x, V(x, 3), label=r"f")
25
26 E = np.full_like(x, -0.4)
27 ax.plot(x, E, "--", label="E", color="cornflowerblue")
28
29 E = np.full_like(x, -0.17)
30 ax.plot(x, E, "--", label="E", color="orange")
31
32 E = np.full_like(x, -0.06)
33 ax.plot(x, E, "--", label="E", color="green")
34
35 E = np.full_like(x, -0.03)
36 ax.plot(x, E, "--", label="E", color="red")
37
38 ax.set_ylim(-0.5, 0.5)
39
40 ax.grid()
41 ax.legend(loc="upper right")
```

C Exercise 9 - Radial Functions 1s, 2s, 3s, 4s, 6s, 9s

Results

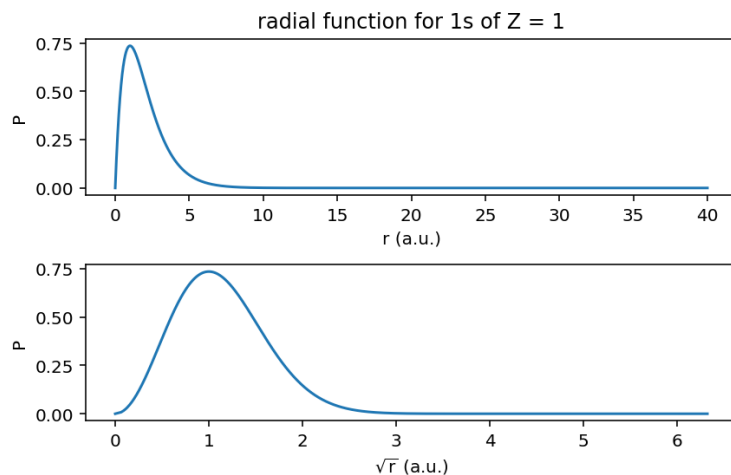


Figure 25: Energy eigenvalue: $E = -4.999 \times 10^{-1}$ a.u.

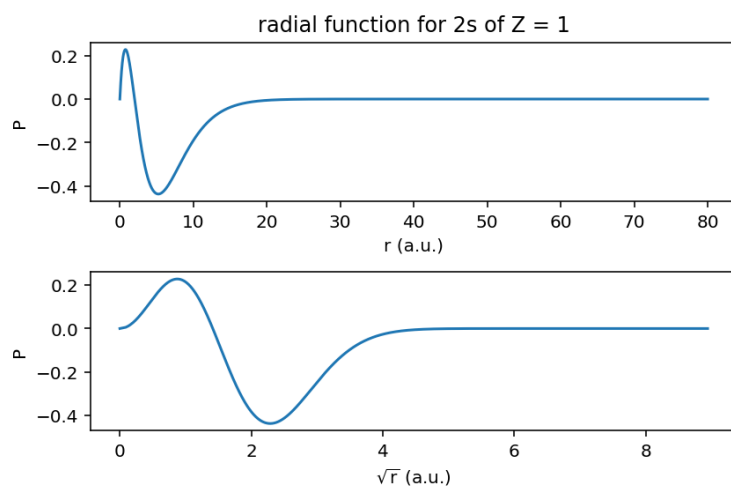


Figure 26: Energy eigenvalue: $E = -1.249 \times 10^{-1}$ a.u.

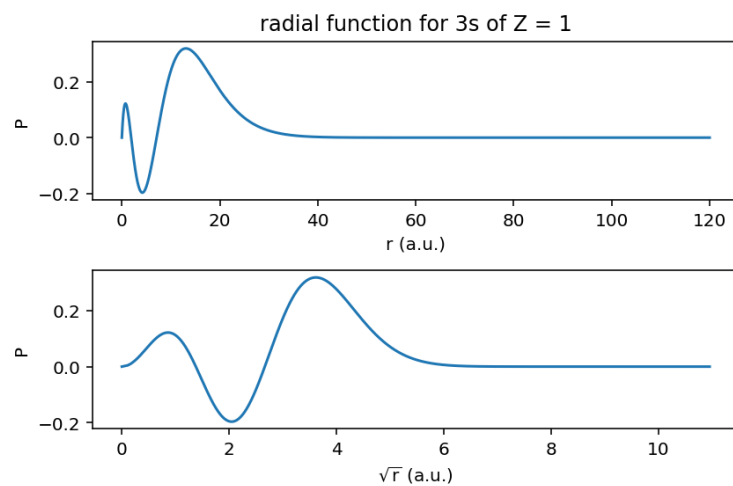


Figure 27: Energy eigenvalue: $E = -5.555 \times 10^{-2}$ a.u.

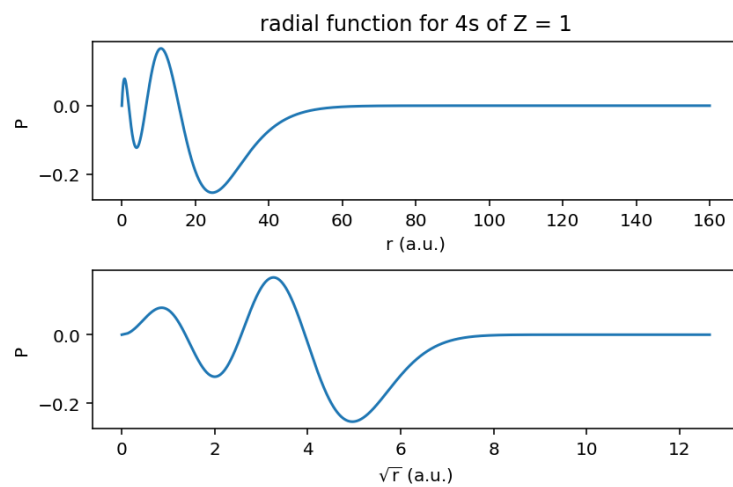


Figure 28: Energy eigenvalue: $E = -3.124 \times 10^{-2}$ a.u.

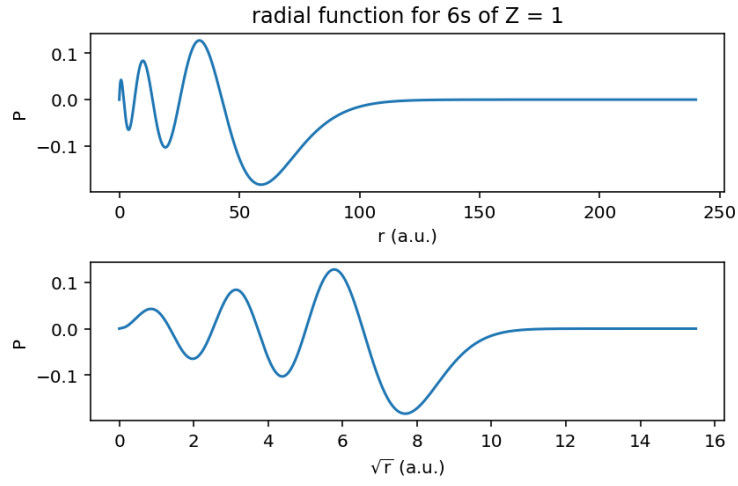


Figure 29: Energy eigenvalue: $E = -1.388 \times 10^{-2}$ a.u.

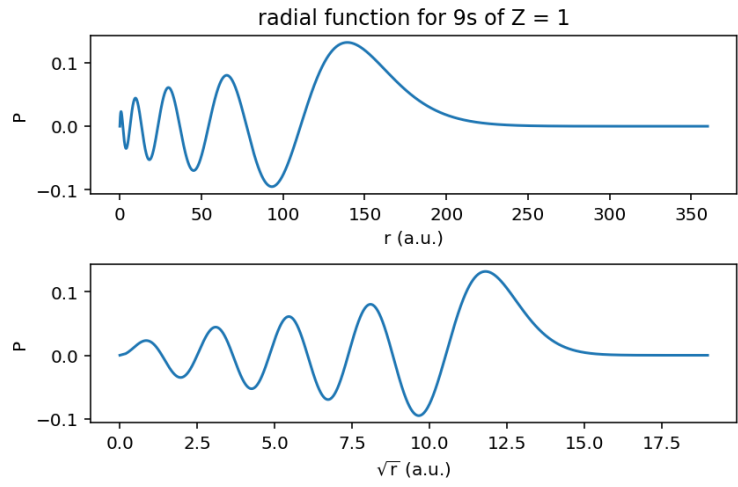


Figure 30: Energy eigenvalue: $E = -6.172 \times 10^{-3}$ a.u.

D Exercise 12 - Radial Functions 1s, 2s, 3s, 4s, 6s, 9s from *radiolog.py*

Results

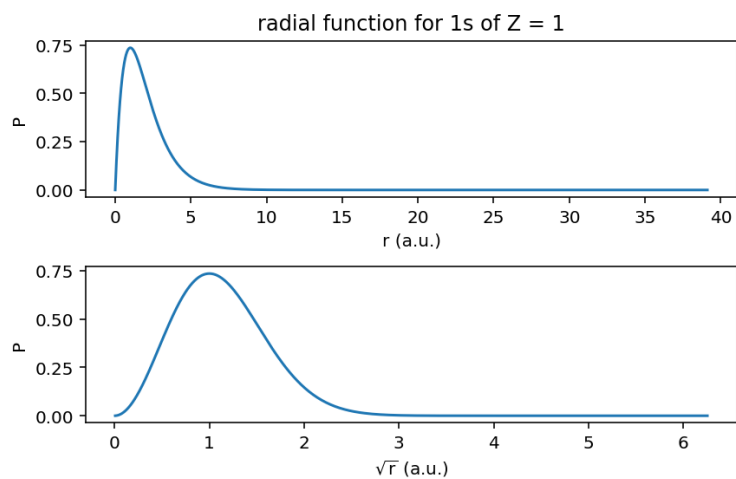


Figure 31: Energy eigenvalue: $E = -5 \times 10^{-1}$ a.u.

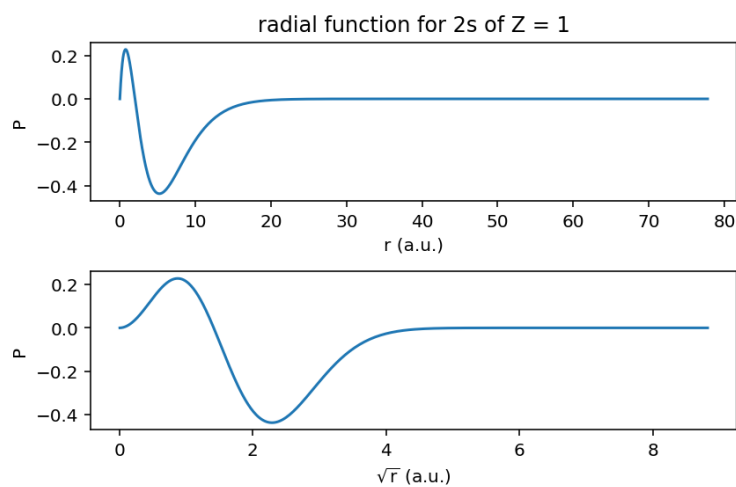


Figure 32: Energy eigenvalue: $E = -1.25 \times 10^{-1}$ a.u.

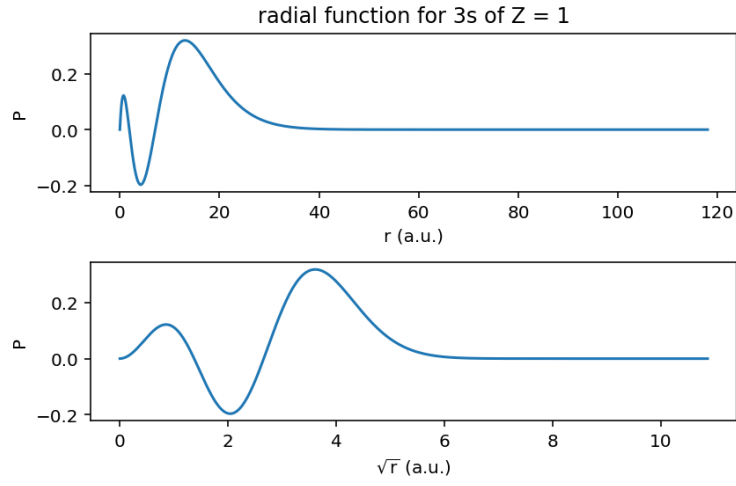


Figure 33: Energy eigenvalue: $E = -5.555 \times 10^{-2}$ a.u.

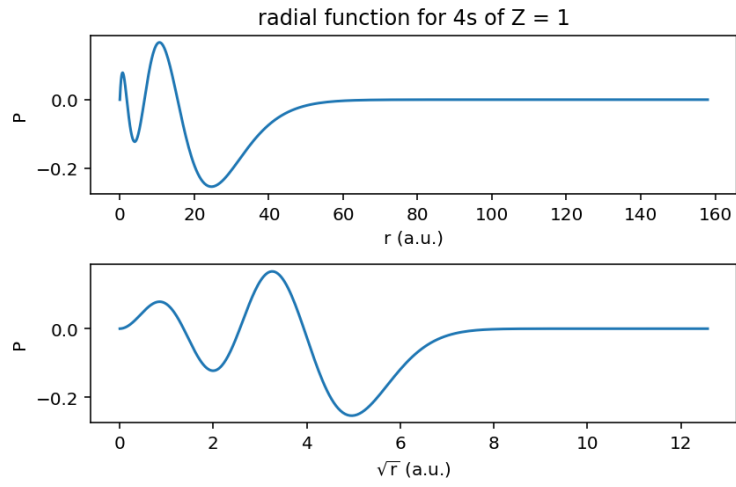


Figure 34: Energy eigenvalue: $E = -3.125 \times 10^{-2}$ a.u.

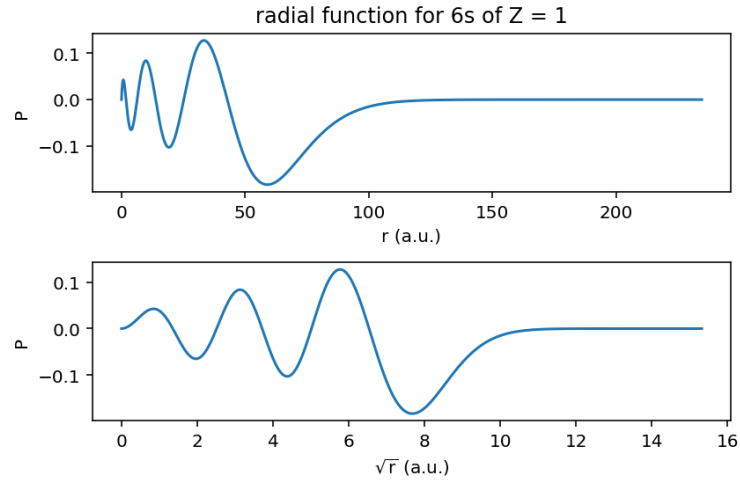


Figure 35: Energy eigenvalue: $E = -1.388 \times 10^{-2}$ a.u.

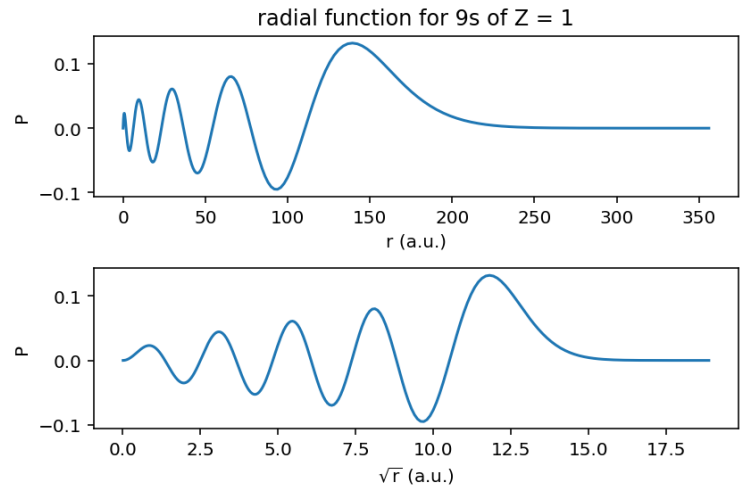


Figure 36: Energy eigenvalue: $E = -6.172 \times 10^{-3}$ a.u.

E Exercise 16 - Radial Orbitals 1s, 2s, 3s, 2p, 3p, 3d and radial expectation *radiolog.py*

Results

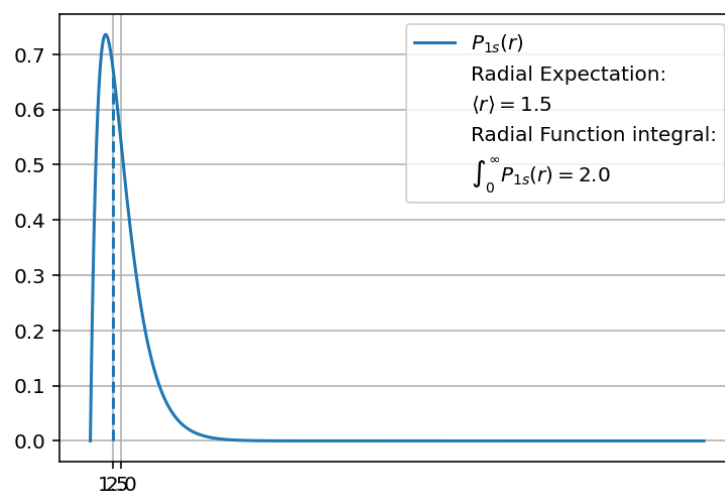


Figure 37: Radial expectation for $P_{1s}(r)$.

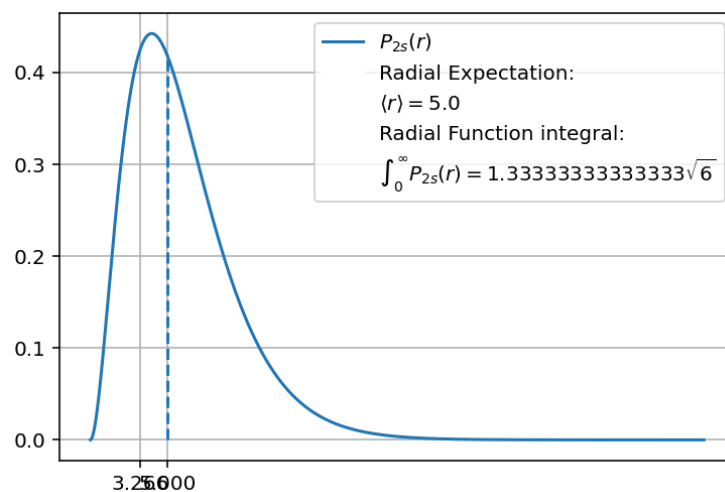


Figure 38: Radial expectation for $P_{2s}(r)$.

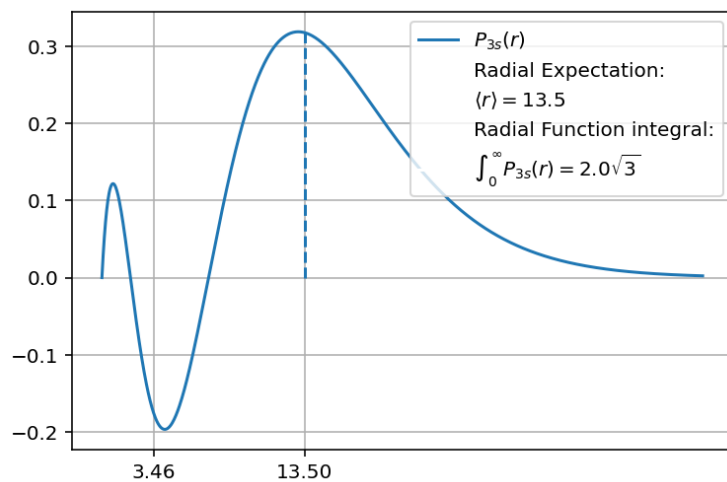


Figure 39: Radial expectation for $P_{3s}(r)$.

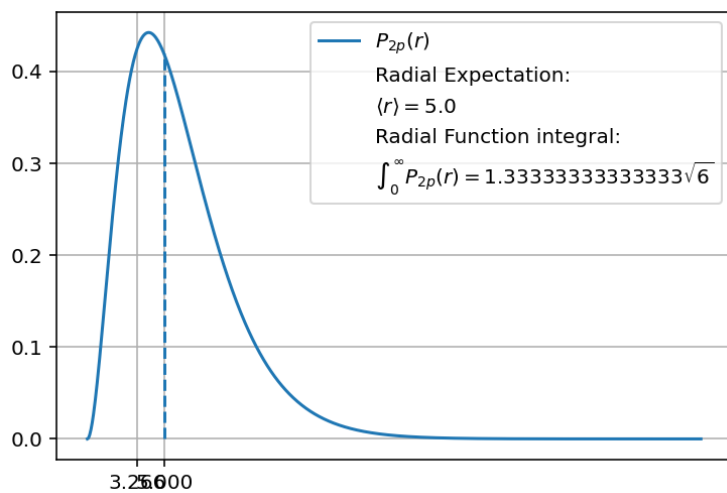


Figure 40: Radial expectation for $P_{3p}(r)$.

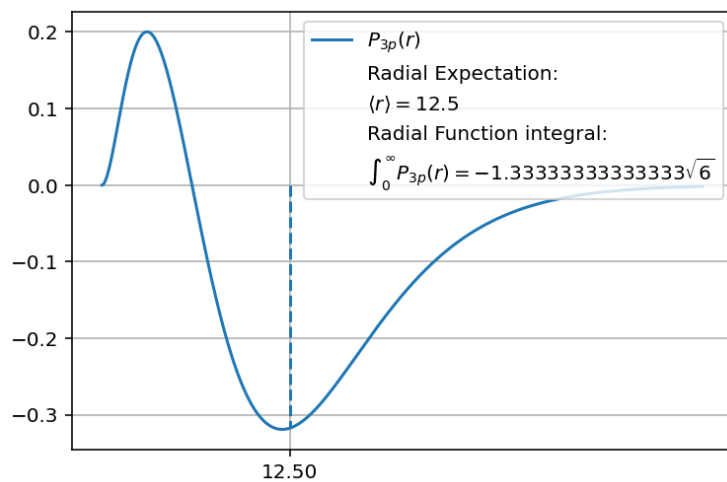


Figure 41: Radial expectation for $P_{3p}(r)$.

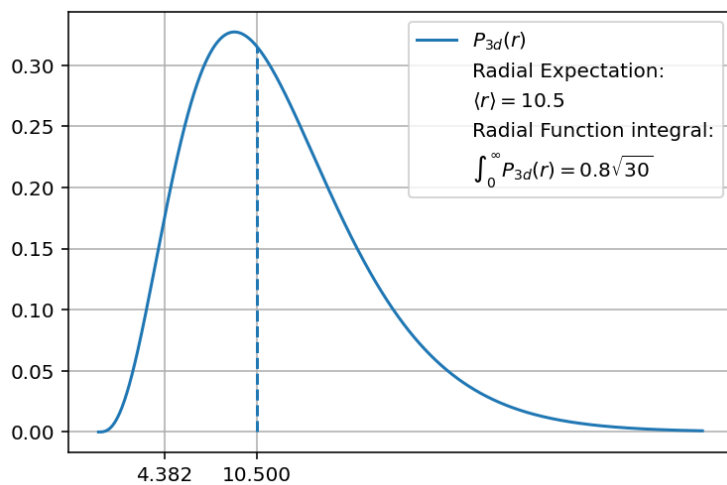


Figure 42: Radial expectation for $P_{3d}(r)$.

F Exercises 19, 20, 21 - Modified radiolog.py using parametric potential

```
1
2 import math
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import sys
6
7
8 def Settings_Al_III():
9     # Program settings fot Al III
10    referenceStates = {
11        "3s": (3, 0, 0),
12        "3p": (3, 1, 53682.93),
13        "3d": (3, 2, 115956.21),
14        "4s": (4, 0, 126164.05),
15        "4p": (4, 1, 143633.38),
16        "4d": (4, 2, 165786.32),
17        "4f": (4, 3, 167612.75),
18        "5s": (5, 0, 170637.35)
19    }
20    Z = 13
21    N = 11
22    # List of states to calculate
23    states = ["2p^6 3s^1", "2p^6 3p^1", "2p^6 3d^1", "2p^6 4s^1", "2p^6 4p^1", "2p^6 4d^1", "2p^6 4f^1", "2p^6 5s^1"]
24    return (referenceStates, N, Z, states)
25
26 def Settings_Na_I():
27     # Settings for Na I
28     referenceStates = {
29        "3s": (3, 0, 0),
30        "3p": (3, 1, 16956.17025),
31        "4s": (3, 2, 25739.999),
32        "3d": (4, 0, 29172.837),
33        "4p": (4, 1, 30266.99 ),
34        "5s": (4, 2, 33200.673),
35        "4d": (4, 3, 34548.729),
36        "4f": (5, 0, 34586.92)
37    }
38    Z = 11
39    N = 11
40    states = ["2p^6 3s^1", "2p^6 3p^1", "2p^6 4s^1", "2p^6 3d^1", "2p^6 4p^1", "2p^6 5s^1", "2p^6 4d^1", "2p^6 4f^1"]
41    return (referenceStates, N, Z, states)
42
43 def Settings_Li_I():
44     # Settings for Li I
45     referenceStates = {
```

```

46     "2s": (2, 0, 0),
47     "2p": (2, 1, 14903.66 ),
48     "3s": (3, 0, 27206.12),
49     "3p": (3, 1, 30925.38 ),
50     "3d": (3, 2, 31283.08),
51     "4s": (4, 0, 35012.06),
52     "4p": (4, 1, 36469.55),
53     "4d": (4, 2, 36623.38)
54 }
55 Z = 3
56 N = 3
57 states = ["1s^2 2s^1", "1s^2 2p^1", "1s^2 3s^1", "1s^2 3p^1", "1s^2 3d
58 ^1", "1s^2 4s^1", "1s^2 4p^1", "1s^2 4d^1"]
59 return (referenceStates, N, Z, states)
60
61
62 def AUnitToSpectroscopic(E):
63     return E*219474.63
64
65 def parametricPotential(r, Z, N, param):
66     return -Z/r + ((N - 1)*r)/(param**2 + r**2)
67
68
69 def radiallog(n,l, N, Z, param, plot=True):
70
71
72     if Z > 0 and l < n:
73         E = -Z**2 / 2 # starting energy taken as the lowest possible
74         nodes = n - l - 1 # targeted number of nodes
75     else:
76         sys.stderr.write('Invalid input\n')
77         sys.exit(-1)
78
79     rhomin = -10
80     h = 1 / 48 # step size
81
82     nodes_count = -1 # initial values of nodes
83     dE = math.inf # initial values of difference of derivat at r_c
84     E_upper = -2.0e-52 # initial values of upper bound of energy during
85     iter
86     E_lower = -math.inf # initial values of lower bound of energy during
87     iter
88     num_iter = 0
89
90     while nodes != nodes_count or abs(dE) > 1.0e-10:
91         # Make sure that the energy is such that the turning point is in
92         the interior of the grid
93         while 1:
94             r_inf = 40 / math.sqrt(2 * abs(E)) # practical infinity
95             rhoi = math.log(Z * r_inf) # practical infinity in rho

```

```

93     grid_points = round((-rhomin + rhoi) / h) # number of grid
points
94     rho = rhomin + np.linspace(0, grid_points - 1, grid_points) *
h # generate grid
95     r = np.exp(rho) / Z
96
97     # Define the effective potential
98     U = np.linspace(0, 0, grid_points)
99     U[0] = 0 # initialize U, gets a 0 value at r = 0
100    U[1:] = parametricPotential(r[1:], Z, N, param) + 1 * (1 + 1)
/ (2 * r[1:]**2)
101 #    U[1:] = -Z / r[1:] + 1 * (1 + 1) / (2 * r[1:]**2)
102
103    # Determine the outer classical turning point. Start from the
practical infinity
104    # and step inwards until U(i) < E
105    i_c = grid_points - 1
106
107    for i in range(grid_points - 1, -1, -1):
108        if U[i] < E:
109            i_c = i
110            break
111
112    # If turning point not in the interior increase the energy and
try again
113    if i_c == grid_points - 1:
114        E = 0.9 * E
115    else:
116        break
117
118    g = -2 * r**2 * (E - parametricPotential(r, Z, N, param)) + (1 + 1
/ 2)**2
119    alpha = 1 - (h**2 / 12) * g # alpha and g function for Numerow
120    beta = 2 + (5 * h**2 / 6) * g # beta for Numerov's method
121
122    # Perform the outward integration
123    P_out = np.zeros(grid_points) # initialize P_out
124    P_out[0] = r[0]**(1 + 1) * (1 - Z * r[0] / (1 + 1)) / math.sqrt(r
[0]) # starting values from regular
125    P_out[1] = r[1]**(1 + 1) * (1 - Z * r[1] / (1 + 1)) / math.sqrt(r
[1]) # solution at r = 0
126
127    for i in range(1, i_c + 1):
128        P_out[i + 1] = (beta[i] * P_out[i] - alpha[i - 1] * P_out[i -
1]) / alpha[i + 1]
129
130    # Perform the inward integration
131    P_in = np.zeros(grid_points) # initialize P_in
132
133    # starting values from regular solution at r_inf
134    P_in[grid_points - 1] = np.exp(-np.sqrt(2 * abs(E) * r[grid_points

```



```

- 1])) / math.sqrt(r[grid_points - 1])
135     P_in[grid_points - 2] = np.exp(-np.sqrt(2 * abs(E) * r[grid_points
- 2])) / math.sqrt(r[grid_points - 2])
136
137     for i in range(grid_points - 2, i_c - 1, -1):
138         P_in[i - 1] = (beta[i] * P_in[i] - alpha[i + 1] * P_in[i + 1])
/ alpha[i - 1]
139
140     # Scale and merge solutions
141     P_out = P_out / P_out[i_c]
142     P_in = P_in / P_in[i_c]
143     P = np.append(P_out[0:i_c], P_in[i_c:])
144
145     # Count the number of nodes. At a node position, P(i+1)P(i) is
negative
146     nodes_count = 0
147
148     for i in range(1, grid_points - 1):
149         if P[i + 1] * P[i] < 0:
150             nodes_count += 1
151
152     # Check nodes and adjust E
153     if nodes_count > nodes:
154         E_upper = min(E_upper, E)
155
156         if E * 1.1 < E_lower:
157             E = (E_lower + E) / 2
158         else:
159             # too many nodes, decrease E
160             E = E * 1.1
161
162     elif nodes_count < nodes:
163         E_lower = max(E_lower, E)
164
165         if E * 0.9 > E_upper:
166             E = (E + E_upper) / 2
167         else:
168             # too few nodes, increase E
169             E = E * 0.9
170
171     # Fine tune the energy
172     if nodes_count == nodes:
173         # Compute the derivatives and take difference
174         DP_out = (P_out[i_c + 1] - P_out[i_c - 1]) / (2 * h)
175         DP_in = (P_in[i_c + 1] - P_in[i_c - 1]) / (2 * h)
176         dE = (DP_out - DP_in) * P[i_c] / (2 * sum(r**2 * P**2) * h)
177
178         # Damp if step is too large
179         while E + dE > 0:
180             dE = dE / 2
181

```

```

182         E = E + dE
183
184
185         num_iter += 1
186
187         # Normalize and check slope
188         norm = math.sqrt(sum(P**2 * r**2) * h)
189         P = P / norm
190
191         if P[1] < 0:
192             P = -P
193
194         # Compute radial expectation value
195     #     r1 = sum(r**3 * P**2) * h
196
197     #     endtime = datetime.datetime.now().timestamp()
198     #     print("time used: %f s" % (endtime - starttime))
199
200     # Write energy and plot solution
201     #     print("Number of iterations: %d" % (num_iter,))
202     #     print("Nodes: %d" % (nodes_count))
203     #     print("Number of grid points: %d" % (grid_points,))
204     #     print("Energy eigenvalue: %.16e a.u.." % (E,))
205     #     print("Radial expectation value: %.16e a.u." % (r1,))
206
207     orbital = ["s", "p", "d", "f", "g", "h", "i"]
208
209     if plot:
210         plt.figure()
211
212         plt.subplot(211)
213         plt.plot(r, np.sqrt(r) * P)
214         plt.xlabel("r (a.u.)")
215         plt.title("radial function for %d%s of Z = %d" % (n, orbital[1], Z
216 ,))
217         plt.ylabel("P")
218
219         rr = np.sqrt(r)
220
221         plt.subplot(212)
222         plt.plot(rr, np.sqrt(r) * P)
223         plt.xlabel('$\sqrt{\mathrm{r}}$ (a.u.)')
224         plt.ylabel("P")
225
226         plt.tight_layout(pad=1.0)
227         plt.show()
228
229     return (r, P, E)
230
231 #

```

```

=====
232 # The ground congura- tion is 1s22s22p63s and the excited congurations are
      given by 1s22s22p63p,
233 # 1s22s22p63d etc
234 #
=====

235
236 def CalculateState(nl, N, Z, param):
237
238     plt.figure(figsize=(12, 9), dpi=200)
239     plt.suptitle(r"Orbital plots", fontsize=24)
240
241     results = []
242
243     letters = {0: "s", 1: "p", 2: "d", 3: "f"}
244
245     for (n, l) in nl:
246
247         r, P, E = radiallog(n, l, N, Z, param, False)
248         results.append((r, P, E))
249
250         label = f"{n}{letters[l]}"
251
252         # Top plot
253
254         plt.subplot(211)
255
256         plt.plot(
257             r,
258             np.sqrt(r) * P,
259             linewidth=1.0,
260             label=label
261         )
262
263         plt.xlim(0, 80)
264         plt.ylim(-1, 1)
265
266
267         plt.xlabel("r (a.u.)", fontsize=18)
268         plt.ylabel(r"$\sqrt{r}P(r)$", fontsize=18)
269
270         plt.legend(
271             loc="upper center",
272             ncols=6,
273             fontsize=18,
274             frameon=False
275         )
276
277         # Bottom plot

```

```

278     rr = np.sqrt(r)
279
280     plt.subplot(212)
281
282     plt.plot(
283         rr,
284         np.sqrt(r) * P,
285         linewidth=1.0,
286         label=label
287     )
288
289     plt.xlim(0, 8)
290
291     plt.xlabel(r"$\sqrt{r}$ (a.u.)", fontsize=18)
292     plt.ylabel(r"$\sqrt{r}P(r)$", fontsize=18)
293
294     plt.legend(
295         loc="upper center",
296         ncols=6,
297         fontsize=18,
298         frameon=False
299     )
300
301
302     plt.tight_layout(rect=[0, 0, 1, 0.96])
303
304     plt.show()
305
306     return results
307
308
309 def FindAlphaParameter(Eref, N, Z):
310     levels = []
311     alpha = 0.1
312     alphaLast = alpha
313     meanEref = 0
314     meanDeltE = 0
315     error = np.inf
316
317     for (n, l, dltaE) in list(Eref.values()):
318         E = radiallog(n, l, N, Z, alpha, False)[2]
319         E = AUnitToSpectroscopic(E)
320         levels.append(E)
321         # Calculate the mean of the reference values
322         meanEref = meanEref + dltaE/len(Eref.values())
323     meanDeltE = (np.array(levels) - levels[0]).mean()
324
325     error = meanEref - meanDeltE
326
327     # Look for a better value for alpha until the error is less than the
    arbitrary tolerance

```

```

328     while(abs(error) > 100):
329         # Adjust the parameter in the contrary direction to the error,
relative to the mean of the reference values
330         alpha = alpha + alpha*(-error)/abs(meanERef)
331
332         # Reset values for new round
333         levels = []
334         alphaLast = alpha
335
336         for (n, l, dltaE) in list(Eref.values()):
337             E = radiallog(n, l, N, Z, alpha, False)[2]
338             E = AUnitToSpectroscopic(E)
339             levels.append(E)
340         meanDeltE = (np.array(levels) - levels[0]).mean()
341
342         # If we obtained some improvement on this round, we update the
parameter and try again
343         error = meanDeltE - meanERef
344
345
346         # The last update to the parameter made the error worse, so we
rollback the change
347         alpha = alphaLast
348         return alpha
349
350
351 if __name__ == '__main__':
352
353     (referenceStates, N, Z, states) = Settings_Li_I()
354
355
356     # Calculate alpha for the parametric potential
357     alpha = FindAlphaParameter(referenceStates, N, Z)
358     print("Optimized value for alpha: %f" %alpha)
359
360     nl = {
361         "1s": (1, 0),
362         "2s": (2, 0),
363         "2p": (2, 1),
364         "3s": (3, 0),
365         "3p": (3, 1),
366         "3d": (3, 2),
367         "4s": (4, 0),
368         "4p": (4, 1),
369         "4d": (4, 2),
370         "4f": (4, 3),
371         "5s": (5, 0)
372     }
373     levels = CalculateState(list(nl.values()), N, Z, alpha)
374
375     for i in range(0, len(nl)):

```

```

376     print("Energy for level %s: %.16e a.u." %(list(nl.keys())[i],
377         levels[i][2]))
378
379     Energies = []
380     for s in states:
381         E = 0
382         orbitals = s.split(' ')
383         for o in orbitals:
384             orbital, electrons = o.split('^')
385             index = list(nl.keys()).index(orbital)
386             E = E + levels[index][2] * int(electrons)
387         Energies.append(E)
388     print(fr"DltaE_{orbitals}=%f cm^-1" %(AUnitToSpectroscopic(E -
389         Energies[0])))

```

References

- [JE22] Jönsson and Jörgen Ekman. Computational physics project i: Atomic structure. *Faculty of Technology and Society, Malmö University*, page 31, 2022.
- [Wik] Wikipedia. Wave interference. https://en.wikipedia.org/wiki/Wave_interference.